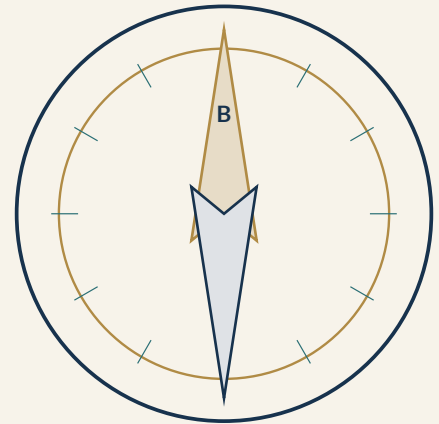


PHASE B IMPLEMENTATION ROADMAP

SIX-PHASE DELIVERY PLAN



Vision Waypoints, Native Capture, Visual Verification, Story Integration, Desktop Application Delivery, and Public-Creator Hardening

Document ID:	TT-VISION-PHASE-B-001
Version:	1.0
Status:	Authorized Implementation Baseline
Date:	July 17, 2026
Parent Specification:	TT-VISION-GOV-001, Vision Waypoint and External AR Governing Specification
Phase Ceiling:	Six top-level implementation phases
Primary Use:	Codex implementation planning, execution control, testing, and release acceptance

Contents

Document Authority and Relationship to the Governing Specification	xii
Normative language	xii
Required companion documents	xii
Document Control	xiv
Executive Summary	xv
1 Phase B Product Boundary	1
1.1 Definition of Phase B complete	1
1.2 Explicit Phase B non-goals	1
1.3 Phase B completion demonstration	2
2 Governing Delivery Principles	3
2.1 Vertical value in every phase	3
2.2 Shared product, not parallel rewrites	3
2.3 Contracts before hidden coupling	3
2.4 Mandatory evidence gates	3
2.5 Creator usability is a release requirement	3
2.6 Local-first privacy	3
2.7 No game modification	3
2.8 Phase exit gates are binary	4
3 Overall System Architecture	5
3.1 Logical components	5
3.2 Canonical application layers	5
3.3 Recommended repository shape	6
4 Website, PWA, and Desktop Strategy	7
4.1 The product shall use three delivery surfaces	7
4.1.1 Browser website	7
4.1.2 Progressive Web App	7
4.1.3 Windows desktop application	7
4.2 One codebase, multiple shells	8
4.3 Delivery comparison	8
4.4 Desktop application framework recommendation	9
4.5 Offline model	9
5 Six-Phase Roadmap at a Glance	10

6	Cross-Phase Program Controls	11
6.1	Top-level phase ceiling	11
6.2	Branching and integration posture	11
6.3	Phase implementation prompt	11
6.4	Completion report	12
6.5	Definition of a prohibited shortcut	12
7	Phase B-1: Shared Platform Foundation	13
7.1	Phase mission	13
7.2	Why this phase exists	13
7.3	In scope	13
7.4	Out of scope	14
7.5	B-1 work package 1: Repository and shared-package architecture	14
7.5.1	Objective	14
7.5.2	Required actions	14
7.5.3	Required dependency direction	14
7.5.4	Expected deliverables	15
7.6	B-1 work package 2: Vision Waypoint domain model	15
7.6.1	Required entities	15
7.6.2	Identity rules	15
7.6.3	Version rules	15
7.7	B-1 work package 3: Lifecycle and state machines	16
7.7.1	Authoring lifecycle	16
7.7.2	State-transition requirements	16
7.7.3	Story-use lifecycle	16
7.8	B-1 work package 4: Database migrations and persistence services	17
7.8.1	Requirements	17
7.8.2	Migration verification	17
7.9	B-1 work package 5: API and event contracts	17
7.9.1	Contract families	17
7.9.2	Companion protocol envelope	18
7.9.3	Required protocol behaviors	18
7.10	B-1 work package 6: Platform adapters	18
7.11	B-1 work package 7: Desktop-shell bootstrap	18
7.11.1	Required behavior	18
7.11.2	Prohibited behavior	19
7.12	B-1 work package 8: PWA foundation	19
7.13	B-1 work package 9: Feature flags and environment control	19
7.14	B-1 work package 10: Test harness and fixtures	20
7.15	B-1 required demonstration	20
7.16	B-1 exit gate	20
7.16.1	Mandatory acceptance checklist	20
7.17	B-1 principal risks and mitigations	21
7.18	B-1 Codex completion evidence	21
8	Phase B-2: Native Companion and Capture Runtime	22
8.1	Phase mission	22
8.2	Why capture is a separate phase	22
8.3	In scope	22
8.4	Out of scope	23

8.5	B-2 work package 1: Native Companion process architecture	23
8.5.1	Recommended process model	23
8.5.2	Process-isolation requirements	23
8.6	B-2 work package 2: Capture-target selection	23
8.6.1	Required behavior	23
8.6.2	Source-change behavior	24
8.7	B-2 work package 3: Creator recording mode	24
8.7.1	Recording session metadata	24
8.7.2	Creator controls	24
8.8	B-2 work package 4: Player hold-to-scan mode	25
8.8.1	Interaction contract	25
8.8.2	Required scan behavior	25
8.9	B-2 work package 5: Global hotkey and input safety	25
8.9.1	Requirements	25
8.10	B-2 work package 6: Ring buffer and frame lifecycle	26
8.10.1	Ring-buffer intent	26
8.10.2	Frame lifecycle rules	26
8.10.3	Resource limits	26
8.11	B-2 work package 7: Frame normalization	26
8.12	B-2 work package 8: Capture-quality service	27
8.13	B-2 work package 9: Best-frame selection	27
8.14	B-2 work package 10: Browser pairing and localhost security	27
8.14.1	Pairing flow	27
8.14.2	Security requirements	28
8.15	B-2 work package 11: Desktop-integrated mode	28
8.16	B-2 work package 12: Capability and hardware discovery	28
8.17	B-2 work package 13: Privacy and consent surface	29
8.18	B-2 work package 14: Failure and recovery model	29
8.19	B-2 work package 15: Observability and diagnostic bundle	30
8.20	B-2 testing strategy	30
8.20.1	Automated tests	30
8.20.2	Manual test matrix	30
8.21	B-2 required demonstration	31
8.22	B-2 exit gate	31
8.22.1	Mandatory acceptance checklist	31
8.23	B-2 principal risks and mitigations	31
8.24	B-2 Codex completion evidence	32
9	Phase B-3: Vision Waypoint Studio Authoring	33
9.1	Phase mission	33
9.2	Creator experience objective	33
9.3	In scope	33
9.4	Out of scope	34
9.5	B-3 work package 1: Vision Waypoint Library	34
9.5.1	Library placement	34
9.5.2	Waypoint card information	34
9.5.3	Library actions	34
9.5.4	Filters and search	35
9.6	B-3 work package 2: New-waypoint purpose selection	35
9.6.1	Area Arrival	35

9.6.2	Exact Landmark	35
9.6.3	Viewpoint	35
9.6.4	Object Inspection	35
9.6.5	Item Pickup	36
9.6.6	Sequence	36
9.6.7	Advanced	36
9.6.8	Type-specific guidance	36
9.7	B-3 work package 3: Story-purpose and completion actions	36
9.8	B-3 work package 4: Companion connection step	36
9.9	B-3 work package 5: Target recording workflow	37
9.9.1	Creator instruction	37
9.9.2	Live guidance	37
9.9.3	Coverage categories	37
9.10	B-3 work package 6: Recording review and curation	38
9.11	B-3 work package 7: Accepted player region	38
9.11.1	Guided walk mode	38
9.11.2	Detailed editor mode	38
9.11.3	Rule presentation	39
9.12	B-3 work package 8: Boundary and invalid-pose recording	39
9.13	B-3 work package 9: Hard-negative recording workflow	39
9.13.1	Required creator task	39
9.13.2	Negative categories	39
9.13.3	Negative requirements by profile	40
9.14	B-3 work package 10: Stable and ignored region editor	40
9.14.1	Purpose	40
9.14.2	Suggested important regions	40
9.14.3	Suggested ignored regions	40
9.14.4	Editor behavior	41
9.15	B-3 work package 11: Data-health dashboard	41
9.16	B-3 work package 12: Resumable wizard and draft safety	41
9.17	B-3 work package 13: Build-input package	42
9.18	B-3 work package 14: Test workflow foundation	42
9.19	B-3 work package 15: Creator modes	43
9.19.1	Guided mode	43
9.19.2	Detailed mode	43
9.19.3	Engineering mode	43
9.20	B-3 work package 16: Accessibility and presentation quality	43
9.21	B-3 testing strategy	44
9.21.1	Automated	44
9.21.2	Human usability	44
9.22	B-3 required demonstration	44
9.23	B-3 exit gate	45
9.23.1	Mandatory acceptance checklist	45
9.24	B-3 principal risks and mitigations	45
9.25	B-3 Codex completion evidence	46
10	Phase B-4: Verification Build and Runtime Engine	47
10.1	Phase mission	47
10.2	Core engineering rule	47
10.3	In scope	47

10.4	Out of scope	48
10.5	B-4 work package 1: Engine modularity and replaceable model adapters	48
10.5.1	Architectural objective	48
10.5.2	Required metadata	48
10.6	B-4 work package 2: Build-job orchestration	49
10.6.1	Build states	49
10.6.2	Build requirements	49
10.7	B-4 work package 3: Dataset preparation	49
10.7.1	Frame extraction	49
10.7.2	Dataset categories	49
10.7.3	Duplicate and diversity handling	50
10.8	B-4 work package 4: Global visual place retrieval	50
10.8.1	Purpose	50
10.8.2	Requirements	50
10.8.3	Multi-region retrieval	50
10.8.4	Retrieval gate	51
10.8.5	Unknown-scene rejection	51
10.9	B-4 work package 5: Hard-negative system	51
10.9.1	Negative index	51
10.9.2	Veto behavior	51
10.9.3	Hard-negative mining	51
10.9.4	Safety rule	51
10.10	B-4 work package 6: Local feature extraction	51
10.10.1	Purpose	51
10.10.2	Requirements	52
10.10.3	Primary and secondary methods	52
10.11	B-4 work package 7: Local feature matching	52
10.12	B-4 work package 8: Geometric verification	52
10.12.1	Planar or approximately planar target	52
10.12.2	Full 3D location	53
10.12.3	Robust estimation	53
10.13	B-4 work package 9: Sparse 3D reconstruction	53
10.13.1	Build objective	53
10.13.2	Reconstruction validation	53
10.13.3	Multiple maps	54
10.14	B-4 work package 10: Camera pose localization	54
10.14.1	Runtime process	54
10.14.2	Pose outputs	54
10.14.3	Viewpoint tolerance	54
10.15	B-4 work package 11: Accepted pose-region finalization	54
10.16	B-4 work package 12: Spatial evidence coverage	55
10.16.1	Why raw count is insufficient	55
10.16.2	Coverage methods	55
10.17	B-4 work package 13: Temporal and sequence consistency	55
10.17.1	Multi-frame requirement	55
10.17.2	Consistency checks	55
10.17.3	Example physically plausible sequence	56
10.18	B-4 work package 14: Checkpoint-specific verification modules	56
10.18.1	Exact Landmark	56
10.18.2	Area Arrival	56

10.18.3	Viewpoint	56
10.18.4	Object Inspection	56
10.18.5	Item Pickup	57
10.18.6	Sequence	57
10.19	B-4 work package 15: Outcome and failure model	57
10.19.1	Example failure reasons	57
10.20	B-4 work package 16: Calibration engine	57
10.20.1	Per-waypoint calibration	57
10.20.2	Calibration outputs	58
10.20.3	Profile behavior	58
10.21	B-4 work package 17: Reliability grades	58
10.21.1	Excellent	58
10.21.2	Good	58
10.21.3	Needs Improvement	58
10.21.4	Unsafe	59
10.22	B-4 work package 18: Runtime package format	59
10.22.1	Package properties	59
10.23	B-4 work package 19: Local runtime execution	59
10.23.1	Runtime flow	59
10.23.2	Hardware paths	60
10.24	B-4 work package 20: Runtime performance budgets	60
10.25	B-4 work package 21: Pilot waypoint program	60
10.25.1	Pilot A: Distinct exact landmark	60
10.25.2	Pilot B: Moderate natural scene	60
10.25.3	Pilot C: Difficult confusable scene	60
10.26	B-4 work package 22: Test-dataset discipline	61
10.27	B-4 work package 23: Statistical reliability evidence	61
10.28	B-4 work package 24: Shadow-mode output	61
10.29	B-4 testing strategy	62
10.29.1	Automated component tests	62
10.29.2	Integration tests	62
10.29.3	Adversarial manual tests	62
10.30	B-4 required demonstration	62
10.31	B-4 exit gate	63
10.31.1	Mandatory acceptance checklist	63
10.32	B-4 principal risks and mitigations	63
10.33	B-4 Codex completion evidence	64
II	Phase B-5: Player, Story, and Captain Integration	65
II.1	Phase mission	65
II.2	In scope	65
II.3	Out of scope	65
II.4	B-5 work package 1: Story-block integration	66
II.4.1	Vision Verification block	66
II.4.2	Event separation	66
II.4.3	Required events	66
II.5	B-5 work package 2: Waypoint activation and package preparation	67
II.5.1	Failure handling	67
II.6	B-5 work package 3: Player prompt and interaction design	67
II.6.1	Activation presentation	67

II.6.2	Scan presentation	67
II.6.3	Completion transition	68
II.7	B-5 work package 4: Result-to-guidance mapping	68
II.7.1	Verified	68
II.7.2	Insufficient visual evidence	68
II.7.3	Ambiguous	68
II.7.4	Not at target	69
II.7.5	System error	69
II.8	B-5 work package 5: Exact-once story progression	69
II.8.1	Requirements	69
II.8.2	Suggested transaction sequence	69
II.9	B-5 work package 6: Phase A reveal integration	69
II.9.1	Captured-scene use	70
II.9.2	View-matched asset selection	70
II.10	B-5 work package 7: Captain live dashboard	70
II.10.1	Live status	70
II.10.2	Evidence summary	70
II.10.3	Captain actions	70
II.11	B-5 work package 8: Operation modes	71
II.11.1	Development	71
II.11.2	Shadow	71
II.11.3	Captain Confirmed	71
II.11.4	Automatic	71
II.11.5	Promotion requirements	71
II.12	B-5 work package 9: Manual fallback and recovery	72
II.12.1	Manual approval reasons	72
II.12.2	Requirements	72
II.13	B-5 work package 10: Offline package caching	72
II.13.1	Pre-adventure preparation	72
II.13.2	Offline story behavior	72
II.13.3	Synchronization	72
II.14	B-5 work package 11: Web-paired runtime	73
II.15	B-5 work package 12: Attempt history and field learning	73
II.15.1	Attempt record	73
II.15.2	Improvement actions	73
II.16	B-5 work package 13: Privacy controls during play	74
II.17	B-5 work package 14: Player and Captain notifications	74
II.18	B-5 work package 15: First production waypoint rollout	74
II.19	B-5 testing strategy	74
II.19.1	Automated	75
II.19.2	End-to-end scenarios	75
II.20	B-5 required demonstration	75
II.21	B-5 exit gate	76
II.21.1	Mandatory acceptance checklist	76
II.22	B-5 principal risks and mitigations	76
II.23	B-5 Codex completion evidence	77
12	Phase B-6: Hardening and Public Creator Release	78
12.1	Phase mission	78
12.2	In scope	78

12.3	Out of scope	78
12.4	B-6 work package 1: Complete regression suite	79
12.4.1	Domain and contract regression	79
12.4.2	Capture regression	79
12.4.3	Authoring regression	79
12.4.4	Vision regression	80
12.4.5	Story regression	80
12.5	B-6 work package 2: Reliability and adversarial test program	80
12.5.1	Negative sweep	80
12.5.2	Positive robustness	80
12.5.3	Release reliability targets	81
12.6	B-6 work package 3: Hardware and display compatibility matrix	81
12.6.1	Required classes	81
12.6.2	For each configuration record	81
12.6.3	Capability tiers	81
12.7	B-6 work package 4: Performance hardening	82
12.7.1	Budgets	82
12.7.2	Required optimizations	82
12.8	B-6 work package 5: Installer and application packaging	82
12.8.1	Installer tests	83
12.9	B-6 work package 6: Application signing and provenance	83
12.10	B-6 work package 7: Updater and rollback	83
12.10.1	Update channels	83
12.10.2	Update components	83
12.10.3	Updater requirements	84
12.10.4	Model update policy	84
12.11	B-6 work package 8: Runtime package security	84
12.11.1	Requirements	84
12.12	B-6 work package 9: Localhost and native security review	84
12.13	B-6 work package 10: Privacy validation	85
12.13.1	Required verification	85
12.13.2	Privacy documentation	85
12.14	B-6 work package 11: Game-safety and policy review	86
12.15	B-6 work package 12: Creator usability validation	86
12.15.1	External creator cohort	86
12.15.2	Unassisted task sequence	86
12.15.3	Observation rule	86
12.15.4	Usability measures	86
12.16	B-6 work package 13: Player and Captain usability validation	87
12.17	B-6 work package 14: Documentation set	87
12.17.1	Creator guide	87
12.17.2	Player guide	87
12.17.3	Captain guide	88
12.17.4	Developer/operator guide	88
12.18	B-6 work package 15: Support and diagnostics	88
12.19	B-6 work package 16: Compatibility and deprecation policy	88
12.19.1	Compatibility dimensions	88
12.19.2	Package statuses	89
12.19.3	Deprecation rules	89
12.20	B-6 work package 17: Release channels and readiness dashboard	89

12.21	B-6 work package 18: Operational monitoring	89
12.22	B-6 work package 19: Stable release procedure	90
12.23	B-6 required demonstration	90
12.24	B-6 exit gate	91
12.24.1	Mandatory acceptance checklist	91
12.25	B-6 principal risks and mitigations	91
12.26	B-6 Codex completion evidence	92
13	Cross-Phase Dependency and Critical Path	93
13.1	Dependency graph	93
13.2	Critical path items	93
14	Phase Scope Control and Anti-Fragmentation Rules	95
14.1	Six phases means six phases	95
14.1.1	Valid decomposition	95
14.1.2	Invalid decomposition	95
14.2	Work-package completion	95
14.3	Deferral rules	96
15	Architecture Decision Record Policy	97
15.1	ADR triggers	97
15.2	ADR minimum template	97
15.3	Model substitutions	98
16	Source Control, Build, and Release Discipline	99
16.1	Canonical source of truth	99
16.2	Commit expectations	99
16.3	Build identification	99
16.4	Generated artifacts	99
17	Testing Pyramid and Evidence Standard	101
17.1	Unit tests	101
17.2	Integration tests	101
17.3	End-to-end tests	101
17.4	Visual and usability verification	102
17.5	Evidence hierarchy	102
18	Reliability Governance	103
18.1	False-positive priority	103
18.2	False-negative mitigation	103
18.3	No universal threshold	103
18.4	Confirmed false accept procedure	103
19	Security and Privacy Governance Across Phases	105
19.1	Threat boundaries	105
19.2	Minimum privacy requirements by phase	105
19.3	No silent scope expansion	105
20	Data and Artifact Ownership	106
20.1	User-owned or project-owned data classes	106
20.2	Retention principles	106

21	Phase-by-Phase Deliverable Matrix	107
22	Master Phase Demonstration Sequence	108
22.1	B-1 demonstration	108
22.2	B-2 demonstration	108
22.3	B-3 demonstration	108
22.4	B-4 demonstration	108
22.5	B-5 demonstration	108
22.6	B-6 demonstration	108
23	Codex Phase Prompt Template	109
23.1	1. Role and authority	109
23.2	2. Current baseline	109
23.3	3. Phase objective	109
23.4	4. In scope	109
23.5	5. Out of scope	109
23.6	6. Non-negotiable architecture	109
23.7	7. Implementation sequence	110
23.8	8. Test requirements	110
23.9	9. Required artifacts	110
23.10	10. Prohibited shortcuts	110
23.11	11. Completion response	110
24	Completion Report Template	111
25	Risk Register	112
26	Phase Metrics	115
26.1	B-1 metrics	115
26.2	B-2 metrics	115
26.3	B-3 metrics	115
26.4	B-4 metrics	115
26.5	B-5 metrics	116
26.6	B-6 metrics	116
27	Post-Phase-B Roadmap Boundary	117
27.1	Phase C candidate: Automatic passive verification	117
27.2	Phase D candidate: View-matched external AR	117
27.3	Phase E candidate: Live surface anchoring	117
27.4	Phase F candidate: Held-object transformation	117
28	Appendix A: Canonical Domain Entity Summary	118
28.1	VisionWaypoint	118
28.2	VisionWaypointVersion	118
28.3	WaypointRecording	119
28.4	WaypointPoseRegion	119
28.5	WaypointBuildJob	119
28.6	WaypointVerificationAttempt	120
29	Appendix B: Canonical Companion Message Catalog	121

30	Appendix C: Verification Result Example	122
31	Appendix D: Runtime Package Manifest Example	124
32	Appendix E: Phase Readiness Checklist	125
32.1	Before beginning B-1	125
32.2	Before beginning B-2	125
32.3	Before beginning B-3	125
32.4	Before beginning B-4	125
32.5	Before beginning B-5	126
32.6	Before beginning B-6	126
33	Appendix F: Master Definition of Done	127
33.1	Product	127
33.2	Architecture	127
33.3	Reliability	127
33.4	Privacy and safety	127
33.5	Delivery	128
34	Appendix G: Glossary	129
35	Final Governing Statement	130

Document Authority and Relationship to the Governing Specification

Authority

This document is the controlling implementation roadmap for Phase B of the Tall Tale Studio Vision Waypoint system. It is subordinate to, and must be interpreted together with, the Vision Waypoint and External AR Governing Specification, document TT-VISION-GOV-001. The governing specification defines what the product must be. This roadmap defines how Phase B shall be divided, implemented, demonstrated, tested, and accepted in no more than six top-level phases.

Where this roadmap and the governing specification appear to conflict, the governing specification takes precedence unless an approved Architecture Decision Record explicitly amends both documents. Codex and human contributors **MUST NOT** use this roadmap as permission to weaken the governing reliability, privacy, creator-usability, safety, or architectural requirements.

This phase document exists to prevent two recurring implementation failures:

1. decomposing one capability into dozens of artificial phases that are difficult to track and never produce a complete result; and
2. combining too many unrelated responsibilities into one enormous phase that produces attractive surfaces over unfinished architecture.

The selected structure contains exactly six top-level phases. Each phase may contain implementation work packages, issues, pull requests, tests, and corrective tasks. Those subdivisions are not additional phases.

Normative language

- **MUST** / **MUST NOT** indicates a mandatory requirement.
- **SHOULD** / **SHOULD NOT** indicates an expected design choice unless an ADR documents a justified exception.
- **MAY** indicates an optional capability that must not violate mandatory requirements.

Required companion documents

The following documents **SHOULD** remain together in the repository:

- TT-VISION-GOV-001: Vision Waypoint and External AR Governing Specification;
- TT-VISION-PHASE-B-001: this Phase B Implementation Roadmap;
- phase-specific Codex implementation prompts and completion reports;
- Architecture Decision Records;
- schema and protocol documentation generated from source;

- reliability and calibration reports for pilot waypoints;
- security, privacy, and release checklists.

Document Control

Field	Governing value
Project	Tall Tale Studio / Date Nights
Capability	Phase B guided active visual verification
Parent capability	Vision Waypoints and external augmented reality
Current starting state	Phase A passive presentation framework complete
Top-level phases	Six, numbered B-1 through B-6
Primary creator surface	Browser-based Studio shared with desktop build
Primary player surfaces	Browser/PWA plus Windows desktop application
Native capability	Windows Capture Companion and local vision runtime
Initial game	Sea of Thieves
Verification interaction	Player-initiated hold-to-scan
Reliability posture	False-positive-averse, ambiguity-rejecting, guided-retry
Delivery posture	Vertical, demonstrable releases with explicit exit gates
Change control	ADR required for material architecture or product changes

Executive Summary

Phase B will transform the existing passive Tall Tale presentation system into a reusable creator platform capable of visually verifying that a player has reached and inspected the correct place in Sea of Thieves. It will also establish a shared application architecture so the current website can be delivered as a normal web application, an installable Progressive Web App, and a full Windows desktop application without becoming three divergent products.

The complete Phase B result will allow a creator to:

1. open the Studio;
2. create a reusable Vision Waypoint;
3. connect the native Capture Companion;
4. record the correct location from several angles and distances;
5. define where players may stand and what they must face;
6. record similar wrong locations and other hard negatives;
7. review or refine important and ignored visual regions;
8. automatically build a calibrated runtime waypoint package;
9. perform guided positive and negative tests;
10. attach the waypoint to a story block;
11. publish an immutable waypoint version with a Tall Tale;
12. run the Tall Tale through the website, PWA, or desktop application;
13. allow the player to hold an Inspect Surroundings control and scan the scene;
14. verify the scene through several independent mandatory gates;
15. launch the existing Phase A cinematic or story reveal only after reliable verification;
16. recover through guided retries or a Captain override when real-world conditions are uncooperative.

The work is divided as follows:

Phase	Name	Primary outcome
B-1	Shared Platform Foundation	One product architecture shared by web, PWA, desktop, Studio, Player, Captain, and vision contracts
B-2	Native Companion and Capture Runtime	Safe game-window capture, creator recording, player scans, hotkeys, pairing, and privacy behavior
B-3	Vision Waypoint Studio Authoring	A polished wizard and reusable waypoint library usable without code or computer-vision knowledge
B-4	Verification Build and Runtime Engine	Calibrated multi-gate localization and rejection engine with reproducible runtime packages

Phase	Name	Primary outcome
B-5	Player, Story, and Captain Integration	Hold-to-scan gameplay, Phase A reveal triggering, shadow mode, diagnostics, overrides, and offline continuity
B-6	Hardening and Public Creator Release	Reliability, installers, updates, security, usability validation, documentation, and stable release

Roadmap Decision

Phase B SHALL contain no more than these six top-level phases. Internal work packages do not become new phases. A phase is complete only when its exit gate is satisfied with evidence, not when Codex has generated files that appear related to the topic.

Phase B Product Boundary

1.1 Definition of Phase B complete

Phase B is complete when the product supports all of the following in a stable, documented, and repeatable manner:

- reusable Vision Waypoints created within Studio;
- a native Companion capable of selecting and capturing the Sea of Thieves window without touching the game process;
- creator target, boundary, environmental-variation, and hard-negative recordings;
- player-initiated multi-frame hold-to-scan verification;
- build-time frame filtering, feature preparation, reconstruction, calibration, and package generation;
- runtime global retrieval, negative separation, local feature matching, geometry, camera localization, spatial coverage, temporal consistency, checkpoint-specific rules, and ambiguity rejection;
- Player, Captain, Studio, and story-engine integration;
- reliable promotion from development to shadow, Captain-confirmed, and automatic operation;
- web, PWA, and Windows desktop delivery from a shared codebase;
- local processing and no retained player footage by default;
- packaging, installation, updates, compatibility checks, diagnostics, and recovery;
- public-creator usability sufficient that Codex is not required for each waypoint.

1.2 Explicit Phase B non-goals

The following capabilities SHALL remain architecturally possible but are not required to close Phase B:

- continuous recognition without a player-initiated scan;
- full-rate live AR compositing over every captured frame;
- continuously anchored carved text that tracks a moving rock surface;
- photorealistic replacement of a held object inside the game feed;
- automatic generation of cinematic artwork;
- a public marketplace with unrestricted community package execution;
- cross-game generalization beyond explicitly supported adapters;
- cloud-scale model training infrastructure;
- a universal location model for all Sea of Thieves content;
- removal of the Captain fallback;
- modification, injection, memory reading, or control of the game process.

1.3 Phase B completion demonstration

The final demonstration **MUST** show one complete Tall Tale path in which:

1. a Vision Waypoint is created entirely through Studio;
2. the waypoint is built and tested against positives and hard negatives;
3. a published immutable version is attached to a story block;
4. a player launches the Tall Tale in the Windows desktop application or website paired to the Companion;
5. the player holds Inspect Surroundings at the correct location;
6. several frames are captured and verified;
7. the correct event is emitted exactly once;
8. the Phase A cinematic reveal executes;
9. the story advances and survives temporary disconnection;
10. an incorrect similar location is rejected;
11. a difficult legitimate scan produces helpful retry guidance;
12. the Captain can inspect the evidence and recover manually if necessary.

Governing Delivery Principles

2.1 Vertical value in every phase

Every phase **MUST** end with a demonstrable product slice. A phase may contain architecture work, but it may not close with only diagrams, placeholder APIs, static UI, or unexercised schemas.

2.2 Shared product, not parallel rewrites

The website, PWA, and desktop application **MUST** share the same UI, domain logic, story engine, data contracts, and design system wherever the platform permits. Platform-specific behavior **MUST** be isolated behind adapters.

2.3 Contracts before hidden coupling

All communication between the web UI, desktop shell, Capture Companion, vision worker, server, and story engine **MUST** use versioned contracts. Components **MUST NOT** depend on undocumented local files, shared process memory, or accidental implementation details.

2.4 Mandatory evidence gates

Verification **MUST** use mandatory gates. A high score in one subsystem **MUST NOT** compensate for failed geometry, invalid camera position, insufficient negative separation, or temporal inconsistency.

2.5 Creator usability is a release requirement

The creator workflow is not a decorative layer over a developer pipeline. A phase that requires creators to manually arrange model folders, edit JSON, run notebooks, or ask Codex to build each waypoint has failed.

2.6 Local-first privacy

Player frames are transient by default. Runtime inference **SHOULD** occur locally. Raw scans **MUST NOT** be retained or uploaded unless the user explicitly enables a diagnostic or cloud-assisted workflow.

2.7 No game modification

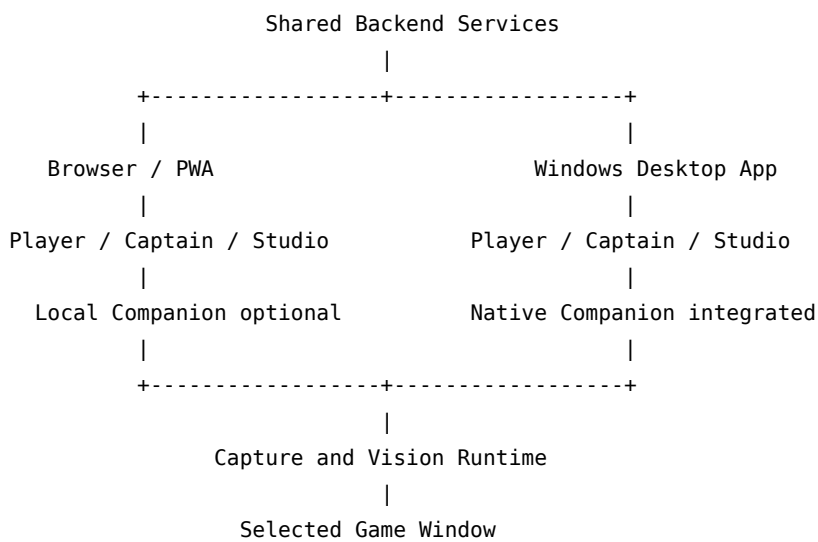
The system observes the user-selected game window externally. It **MUST NOT** inject into the game, read memory, automate controls, modify files, or render an in-game overlay.

2.8 Phase exit gates are binary

A phase is either complete or not complete. A green interface does not compensate for missing backend behavior. A passing unit test does not compensate for a broken end-to-end demonstration. Exit evidence **MUST** include automated tests, human-visible behavior, and updated documentation.

Overall System Architecture

3.1 Logical components



3.2 Canonical application layers

The platform SHOULD be organized into the following logical layers:

1. **Presentation layer** - Player, Captain, Studio, onboarding, settings, and diagnostics;
2. **Story layer** - Tall Tale state machine, block graph, events, progression, and Phase A presentation;
3. **Vision domain layer** - waypoint identity, versions, recordings, build states, calibration, publication, and verification attempts;
4. **Platform-adapter layer** - web Companion connection, desktop native bridge, storage, hotkeys, notifications, and updates;
5. **Capture layer** - native window selection, frame acquisition, buffering, quality measurement, and recording;
6. **Vision build layer** - frame preparation, reference indexing, reconstruction, calibration, and runtime package generation;
7. **Vision runtime layer** - scan processing, mandatory gates, result production, and diagnostics;
8. **Service layer** - accounts, collaboration, synchronization, artifact storage, publication, and audit history.

3.3 Recommended repository shape

The exact current repository layout may be migrated incrementally, but the final architecture SHOULD resemble:

```
apps/  
  web/  
  desktop/  
  server/  
  vision-worker/  
  companion-service/  
  
packages/  
  ui/  
  design-system/  
  story-engine/  
  studio-domain/  
  vision-waypoint-domain/  
  vision-protocol/  
  platform-adapters/  
  package-schemas/  
  test-fixtures/  
  
Development_Docs/  
  AR/  
    Governing_Specification/  
    Phase_B_Roadmap/  
    ADRs/  
    Phase_Completion_Reports/
```

No team is required to rename the existing repository solely to match this example. The governing requirement is separation of concerns and shared packages, not ceremonial folder rearrangement.

Website, PWA, and Desktop Strategy

4.1 The product shall use three delivery surfaces

4.1.1 Browser website

The website remains the universal access surface for:

- account creation and authentication;
- Player, Captain, and Studio access from any supported browser;
- invitation and collaboration flows;
- Tall Tale library browsing;
- non-vision editing;
- mobile and tablet access;
- central deployment and immediate updates.

4.1.2 Progressive Web App

The website SHOULD be installable as a PWA to provide:

- a dedicated app window;
- app icon and launch behavior;
- cached shell and selected content;
- reduced browser chrome;
- lightweight app-like use without a native installer.

The PWA does not replace the native Companion because browser security intentionally limits persistent capture, global hotkeys, and local GPU integration.

4.1.3 Windows desktop application

The desktop application provides the full experience:

- native game-window selection and capture;
- integrated Companion service;
- configurable global hold-to-scan hotkey;
- local GPU and CPU inference;
- system-tray controls;
- cached active Tall Tales and waypoint packages;
- stronger offline operation;
- native diagnostics, installation, updating, and rollback;

- controlled cinematic presentation in a dedicated window.

4.2 One codebase, multiple shells

The desktop application **MUST NOT** become an independent rewrite. The same frontend source **SHOULD** be built for web and desktop. Platform-specific behavior is provided through a narrow adapter interface.

Example contract:

```
interface VisionPlatformAdapter {
  getCapabilities(): Promise<VisionCapabilities>;
  connect(): Promise<CompanionConnection>;
  selectCaptureTarget(): Promise<CaptureTarget>;
  beginCreatorRecording(request: RecordingRequest): Promise<RecordingSession>;
  beginPlayerScan(request: ScanRequest): Promise<VerificationAttempt>;
  pauseVision(): Promise<void>;
  resumeVision(): Promise<void>;
  getDiagnosticBundle(id: string): Promise<DiagnosticBundleMetadata>;
}
```

A web adapter communicates with a paired localhost Companion. A desktop adapter invokes the integrated native service. The Studio and Player interfaces use the same domain operation in both cases.

4.3 Delivery comparison

Capability	Website	PWA	Windows desktop
No installation required	Yes	Initial browser visit	No
Immediate central updates	Yes	Yes, with cache rules	Application update required
Player/Captain/Studio UI	Yes	Yes	Yes
Mobile/tablet access	Yes	Yes	No
Native game-window capture	Companion required	Companion required	Integrated
Global hotkey	Companion required	Companion required	Integrated
Local GPU vision	Companion required	Companion required	Integrated
Strong offline operation	Limited	Moderate	Strong
System tray and capture controls	No	No	Yes
Installer/signing/support burden	None	Low	High
Best use	Universal access	Lightweight installed experience	Full creator/player vision experience

4.4 Desktop application framework recommendation

A lightweight desktop shell that reuses the existing frontend and exposes narrowly permissioned native commands is preferred. Tauri is the current recommended candidate because it permits a shared web frontend with native Rust capabilities and sidecar workers. This is an implementation recommendation, not a permanent schema dependency. An ADR may select another framework if it preserves the same architecture, security boundaries, update behavior, and shared-code requirement.

4.5 Offline model

The desktop application SHOULD cache:

- the current authenticated identity and secure refresh material as permitted;
- active Tall Tale story package;
- required Vision Waypoint runtime packages;
- Phase A images, video, audio, and Three.js assets;
- current progression state;
- pending verification and story events;
- package compatibility metadata.

Offline state MUST reconcile through idempotent, versioned events. The website and desktop application MUST NOT independently advance the same story stage in incompatible ways.

Six-Phase Roadmap at a Glance

Phase	Core question	End-state demonstration
B-1	Can all surfaces share one stable platform and contract system?	A mock waypoint is authored, versioned, verified, and advances a mock story in both web and desktop shells
B-2	Can the product safely observe the selected game window and produce trustworthy scan evidence?	Browser and desktop initiate a real five-second capture scan and receive a structured frame-quality result
B-3	Can a nontechnical creator gather a complete waypoint dataset entirely inside Studio?	A creator records targets, valid regions, and negatives, then exports a valid build-input package without code
B-4	Can the engine build and verify waypoints accurately against unseen positives and dangerous negatives?	Three pilot waypoints run in shadow mode with reproducible calibration and zero accepted locked negatives
B-5	Can verified scans drive the real story safely and beautifully?	Hold-to-scan triggers a Phase A reveal, while Captain diagnostics, retries, offline continuity, and overrides work
B-6	Can another person install, author, publish, and run the system without developer assistance?	External creator usability, hardware matrix, updater, security, documentation, and stable release gates pass

Cross-Phase Program Controls

6.1 Top-level phase ceiling

Only B-1 through B-6 are top-level phases. Work inside a phase is tracked as:

- work packages;
- epics;
- stories;
- issues;
- tasks;
- test cases;
- defects;
- ADRs;
- pull requests.

They **MUST NOT** be renamed as new phases merely because they are substantial.

6.2 Branching and integration posture

Each phase **SHOULD** use a dedicated integration branch or coordinated feature branches with frequent integration into the canonical development branch. Long-lived isolated branches are discouraged because they hide contract drift.

A recommended structure is:

```
development/vision-phase-b
  feature/b1-domain-contracts
  feature/b1-desktop-shell
  feature/b1-pwa
```

After the B-1 gate passes, the phase branch merges and the next phase begins from that verified baseline.

6.3 Phase implementation prompt

Each Codex phase prompt **MUST** include:

1. parent governing-document references;
2. exact phase objective;
3. in-scope work packages;
4. explicit out-of-scope work;
5. repository and branch expectations;

6. migrations and compatibility rules;
7. mandatory tests;
8. required demonstration;
9. exit-gate checklist;
10. completion-report format;
11. prohibition against replacing real behavior with mocks unless the phase explicitly authorizes a mock boundary.

6.4 Completion report

Codex **MUST** produce a phase completion report containing:

- commits and branches;
- files and schemas changed;
- migrations created and applied;
- automated-test results;
- manual demonstration steps and results;
- screenshots or recordings where useful;
- known limitations;
- deferred work mapped to an authorized later phase;
- ADRs created;
- exit-gate evidence;
- rollback instructions.

6.5 Definition of a prohibited shortcut

Examples include:

- creating a static Studio screen without persistence;
- using a random confidence value in place of a vision result;
- embedding desktop-only assumptions directly in shared components;
- saving player scans permanently because temporary storage is easier;
- permitting a high similarity score to override invalid geometry;
- using one screenshot as the complete target dataset;
- hard-coding one pilot rock into the runtime;
- mutating published waypoint versions;
- enabling automatic story progression before shadow validation;
- claiming a phase complete because compilation succeeds.

Phase B-1: Shared Platform Foundation

7.1 Phase mission

Create the shared product architecture, domain model, contracts, persistence, desktop-shell foundation, and delivery boundaries required by every later phase. Phase B-1 SHALL establish one canonical definition of a Vision Waypoint and one canonical application architecture before native capture or computer vision is allowed to accumulate hidden assumptions.

At completion, the web application, installable PWA shell, and Windows desktop development shell will render the same Player, Captain, and Studio interface from shared source. A fully persisted mock Vision Waypoint will flow through authoring, versioning, mock verification, and story progression.

7.2 Why this phase exists

Without B-1, the project is likely to produce:

- one waypoint format in Studio and another in the vision worker;
- a desktop application that copies the website and begins drifting immediately;
- ad hoc localhost messages with no compatibility contract;
- published stories that reference mutable objects;
- capture logic embedded in UI components;
- phase-specific database tables that cannot support future waypoint versions;
- a pile of JSON files with names such as `final_rock_v7_REAL.json`.

B-1 is not bureaucracy. It is the work that prevents every later phase from becoming archaeology.

7.3 In scope

- repository and package boundaries;
- shared UI and domain extraction where needed;
- Vision Waypoint entities and lifecycle;
- immutable version model;
- database migrations;
- API and event contracts;
- platform-adapter interface;
- mock Companion adapter;
- mock verification adapter;
- desktop shell bootstrap;
- PWA manifest and service-worker foundation;

- feature flags;
- test fixtures and contract tests;
- package and protocol versioning;
- governing-document placement and ADR process;
- baseline observability and audit events.

7.4 Out of scope

- real game-window capture;
- real computer-vision inference;
- complete authoring wizard;
- automatic waypoint builds;
- automatic story progression from real evidence;
- public installers or signed releases;
- live AR effects.

Mocks are authorized only at the capture and vision-result boundaries. Domain persistence, lifecycle, versioning, story events, and cross-shell rendering **MUST** be real.

7.5 B-1 work package 1: Repository and shared-package architecture

7.5.1 Objective

Separate shared product capabilities from platform shells without destabilizing the existing working website.

7.5.2 Required actions

1. inventory current Player, Captain, Studio, story, authentication, asset, and routing code;
2. identify components that are already reusable and components tightly coupled to browser globals;
3. define canonical package boundaries;
4. extract shared domain and UI code incrementally;
5. add dependency rules preventing shared packages from importing desktop-only or server-only code;
6. preserve existing routes and user-facing behavior;
7. create import aliases and lint rules to keep boundaries enforceable;
8. document the package ownership and allowed dependency direction.

7.5.3 Required dependency direction

UI components

- > domain interfaces
- > service contracts
- > platform adapter interfaces

Web shell

- > shared UI and web adapters

Desktop shell

- > shared UI and desktop adapters

Native services

- > protocol schemas, not frontend implementation

Shared packages **MUST NOT** import from `apps/web` or `apps/desktop`. Platform shells may import shared packages.

7.5.4 Expected deliverables

- repository architecture document;
- package dependency diagram;
- lint or build checks for invalid imports;
- shared application bootstrap;
- migration plan for remaining legacy code;
- tests proving both shells render the same core routes.

7.6 B-1 work package 2: Vision Waypoint domain model

7.6.1 Required entities

The persistence and domain layer **MUST** support at least:

- `VisionWaypoint` - stable identity and reusable library item;
- `VisionWaypointVersion` - immutable published build lineage and editable draft state;
- `WaypointReferenceRecording` - target, accepted-area, boundary, environmental, or diagnostic recording;
- `WaypointNegativeRecording` - hard-negative recording and confuser classification;
- `WaypointStableRegion` - creator-confirmed important or ignored regions across reference frames;
- `WaypointAcceptedPoseRegion` - accepted and excluded camera regions and orientation rules;
- `WaypointBuildJob` - queued, active, completed, canceled, or failed build;
- `WaypointBuildArtifact` - generated runtime package components and checksums;
- `WaypointCalibrationReport` - thresholds, distributions, datasets, engine versions, and release grade;
- `WaypointTestRun` - positive, negative, boundary, environmental, or shadow test;
- `WaypointVerificationProfile` - Balanced, Strict, Story-Critical, or Custom requirements;
- `WaypointVerificationAttempt` - runtime attempt, outcome, failed gates, and story context;
- `WaypointEvidenceBundle` - temporary or retained diagnostic metadata;
- `WaypointPublication` - publication scope, package version, and compatibility;
- `WaypointUsageReference` - Tall Tales, blocks, drafts, or published stories using the version;
- `CompanionDevice` - paired local device identity and capabilities;
- `CompanionPairingSession` - short-lived pairing authorization.

7.6.2 Identity rules

- A waypoint has a stable ID independent of its name.
- A waypoint version has its own immutable ID.
- Recordings and tests are attached to a specific draft version.
- A published story references a specific published waypoint version.
- Renaming a waypoint does not change its identity.
- Deleting a draft cannot invalidate a published story.
- Archived waypoints remain resolvable when referenced by history.

7.6.3 Version rules

A waypoint **SHALL** support:

Stable Waypoint

Version 1 - Published

Version 2 - Published

Version 3 - Current Draft

Only the current draft may be edited. Publishing freezes:

- runtime package checksum;
- calibration report;
- engine and model versions;
- verification profile;
- accepted pose region;
- stable-region definitions;
- test evidence;
- compatibility metadata.

A published version may be deprecated but not silently changed.

7.7 B-1 work package 3: Lifecycle and state machines

7.7.1 Authoring lifecycle

DRAFT

- > COLLECTING_REFERENCE_DATA
- > READY_TO_BUILD
- > BUILDING
- > NEEDS_ADDITIONAL_DATA | BUILD_FAILED | READY_FOR_TESTING
- > TESTING
- > UNSAFE | SHADOW_READY
- > CAPTAIN_CONFIRMED
- > AUTOMATIC_READY
- > PUBLISHED
- > DEPRECATED | OUTDATED

7.7.2 State-transition requirements

- transitions **MUST** be explicit commands, not inferred from unrelated field values;
- invalid transitions **MUST** fail with a domain error;
- every transition **MUST** create an audit event;
- build and publication operations **MUST** be idempotent;
- the system **MUST** distinguish temporary operational failure from a product-safety failure;
- **UNSAFE** means one or more negative cases were accepted or an equivalent critical reliability failure occurred;
- **OUTDATED** means compatibility metadata indicates the package requires retesting or rebuilding;
- rollback **MUST** restore the previous stable state without mutating historical evidence.

7.7.3 Story-use lifecycle

A story block referencing a waypoint version may be:

UNCONFIGURED

CONFIGURED

PACKAGE_REQUIRED

PACKAGE_READY

ACTIVE

SCANNING

VERIFIED
RETRY_REQUIRED
CAPTAIN_OVERRIDE
COMPLETED

The story state and waypoint-authoring state are separate. A published waypoint may be reused by many active story sessions.

7.8 B-1 work package 4: Database migrations and persistence services

7.8.1 Requirements

- use the project's canonical database migration system;
- use transactional writes for version publication and story attachment;
- enforce unique constraints for stable IDs and version numbers;
- enforce referential integrity between stories and published waypoint versions;
- store large recordings and runtime packages in artifact storage, not as oversized database blobs unless an ADR justifies it;
- store cryptographic hashes and sizes for artifacts;
- preserve user ownership and collaborator permissions;
- support soft archive where historical references exist;
- include created, modified, created-by, and relevant provenance fields;
- define cleanup policies for abandoned draft recordings and expired evidence.

7.8.2 Migration verification

Tests MUST demonstrate:

- migration from the current schema;
- clean database creation;
- rollback or downgrade behavior where the project supports it;
- no loss of existing Tall Tales, Player libraries, Captain state, or Studio assets;
- constraints prevent a published story from referencing an editable draft;
- deletion rules preserve history.

7.9 B-1 work package 5: API and event contracts

7.9.1 Contract families

The platform MUST define versioned schemas for:

- Studio waypoint commands and queries;
- recording-session commands;
- build-job commands and progress;
- test-run commands and results;
- publication and version selection;
- Companion pairing and capabilities;
- player scan requests and progress;
- verification results and evidence metadata;
- story events and idempotency;
- diagnostic bundle requests;
- package download and compatibility checks.

7.9.2 Companion protocol envelope

Every message SHOULD contain:

```
{
  "protocolVersion": "1.0",
  "messageType": "verification.requested",
  "messageId": "msg_...",
  "requestId": "req_...",
  "sessionId": "session_...",
  "sentAt": "2026-07-17T23:00:00-04:00",
  "payload": {}
}
```

7.9.3 Required protocol behaviors

- unknown message types are rejected safely;
- unsupported versions return a compatibility error;
- duplicate request IDs do not duplicate side effects;
- progress events can be lost without corrupting final state;
- final result events are durable or retryable;
- session expiration is explicit;
- payload validation occurs at every process boundary;
- logs may record IDs and outcomes but not raw frame contents by default.

7.10 B-1 work package 6: Platform adapters

The shared application MUST depend on capability interfaces rather than global browser or desktop APIs.

Required adapter families include:

- vision/capture adapter;
- secure-storage adapter;
- file-selection and export adapter;
- notification adapter;
- hotkey-capability adapter;
- offline-package adapter;
- updater-capability adapter;
- platform-information adapter.

The initial web and desktop adapters may return mocked vision outcomes, but their capabilities and error behavior MUST match the future real implementations.

7.11 B-1 work package 7: Desktop-shell bootstrap

7.11.1 Required behavior

- build the existing shared frontend into a desktop development shell;
- route authentication safely through the approved desktop flow;
- isolate native commands behind an allowlisted interface;
- support development and production configuration;
- establish secure local data directories;
- add crash logging without sensitive frame data;

- demonstrate Player, Captain, and Studio navigation;
- include a placeholder Companion status page driven by the adapter.

7.11.2 Prohibited behavior

- loading the production website as an uncontrolled remote page inside a native wrapper;
- exposing arbitrary filesystem or shell access to frontend code;
- duplicating the entire frontend inside the desktop folder;
- embedding secrets in the application bundle;
- allowing arbitrary remote origins to invoke native commands.

7.12 B-1 work package 8: PWA foundation

The web application SHOULD include:

- valid web application manifest;
- app icons and display mode;
- installability checks;
- service-worker foundation;
- versioned cache strategy;
- offline shell and graceful unavailable states;
- explicit separation between cacheable public assets and authenticated/private responses;
- update notification when a new frontend build is available.

Phase B-1 does not require complete offline Tall Tale execution, but the cache architecture MUST not block B-5.

7.13 B-1 work package 9: Feature flags and environment control

Required feature flags:

```
vision_waypoints
vision_waypoint_library
creator_capture
vision_build_engine
player_hold_to_scan
shadow_verification
automatic_vision_progression
desktop_shell
pwa_install
live_external_ar
```

Feature flags MUST be:

- environment-aware;
- auditable;
- safe when disabled;
- unavailable to unauthorized users where appropriate;
- tested in both enabled and disabled states.

7.14 B-1 work package 10: Test harness and fixtures

Create reusable fixtures for:

- draft and published waypoints;
- valid and invalid lifecycle transitions;
- paired and expired Companion sessions;
- mock verification outcomes;
- story blocks with version references;
- package compatibility states;
- duplicate story events;
- archived and deprecated waypoints.

Automated tests **MUST** include:

- domain unit tests;
- schema validation tests;
- database integration tests;
- API contract tests;
- shared UI tests;
- web/desktop adapter parity tests;
- end-to-end mock story progression;
- feature-flag regression tests.

7.15 B-1 required demonstration

The phase demonstration **SHALL** show:

1. the same Studio route in a browser and desktop shell;
2. creation of a persisted mock Vision Waypoint;
3. creation and publication of an immutable mock version;
4. attachment of that version to a story block;
5. a mock five-second verification request through the platform adapter;
6. an idempotent `verification_succeeded` event;
7. story progression and a Phase A placeholder reveal;
8. audit history showing the full sequence;
9. no duplicate advancement when the success event is replayed.

7.16 B-1 exit gate

Phase B-1 Exit Gate

Phase B-1 is complete only when the shared architecture, real domain persistence, immutable versioning, versioned contracts, web/PWA/desktop foundations, platform adapters, mock end-to-end story flow, migrations, tests, and documentation all pass. Real capture and inference may remain mocked; lifecycle, publication, persistence, and story behavior may not.

7.16.1 Mandatory acceptance checklist

- ☐ Existing website behavior has no material regression.
- ☐ Browser and desktop shells render shared Player, Captain, and Studio code.

- ☐ Shared packages do not import platform shells.
- ☐ Vision Waypoint domain entities are migrated and documented.
- ☐ Published versions are immutable.
- ☐ Story blocks reference published versions, not editable drafts.
- ☐ Protocol schemas validate at every boundary.
- ☐ Duplicate verification events do not duplicate story advancement.
- ☐ PWA foundation is installable in a supported browser.
- ☐ Feature flags safely hide unfinished functionality.
- ☐ Mock verification works through both web and desktop adapters.
- ☐ Automated tests and manual demonstration pass.
- ☐ Completion report and ADRs are committed.

7.17 B-1 principal risks and mitigations

Risk	Consequence	Required mitigation
Over-refactoring the working website	Phase stalls before vision work begins	Extract incrementally; preserve routes; use compatibility wrappers
Desktop shell becomes a duplicate app	Immediate product drift	Shared frontend build and adapter boundary enforced by lint/build rules
Schemas designed around one pilot waypoint	Future types require breaking migrations	Model stable identity, versions, generic recordings, rules, and artifacts
Mocks leak into production behavior	False sense of completion	Feature flags, clearly named mock adapters, tests asserting production wiring
Published versions remain mutable	Active stories change unexpectedly	Database constraints, service-layer checks, immutable artifact hashes
Protocol is informal	Later processes become tightly coupled	Versioned schemas and contract tests before native implementation

7.18 B-1 Codex completion evidence

Codex MUST provide:

- architecture and dependency diagrams;
- migration identifiers and schema summary;
- generated or human-readable API schema references;
- screenshots of browser and desktop parity;
- automated test output;
- end-to-end demonstration procedure;
- list of mock boundaries that remain;
- no-regression summary for existing Player, Captain, and Studio behavior;
- rollback steps.

Phase B-2: Native Companion and Capture Runtime

8.1 Phase mission

Build the native, privacy-conscious bridge that can observe the explicitly selected Sea of Thieves window, gather creator recordings, perform player hold-to-scan capture, and communicate with the website or desktop shell through authenticated local contracts. This phase produces trustworthy visual evidence but does not decide whether the scene is correct.

At completion, both the browser/PWA and Windows desktop application will be able to initiate a real capture session, receive frame-quality and scan metadata, and safely stop or recover from capture interruptions.

8.2 Why capture is a separate phase

The capture layer is a security, privacy, reliability, and performance subsystem in its own right. Mixing it immediately with machine-learning decisions makes it difficult to distinguish:

- bad frames from bad models;
- a frozen capture source from a negative visual result;
- a browser pairing failure from an inference failure;
- insufficient scene motion from an incorrect location;
- native-driver instability from story-engine behavior.

B-2 SHALL make capture observable and testable before B-4 is allowed to judge it.

8.3 In scope

- native Windows Companion service;
- desktop integration and browser pairing;
- user-selected game-window capture;
- creator recording sessions;
- player hold-to-scan sessions;
- configurable hotkey;
- transient ring buffer;
- frame normalization;
- capture-quality analysis;
- recording-session metadata;
- privacy controls and indicators;
- capability discovery;
- hardware and performance diagnostics;
- secure localhost protocol;

- failure recovery;
- fake or passthrough verification response for end-to-end testing.

8.4 Out of scope

- global place recognition;
- feature matching and 3D localization;
- waypoint build pipeline;
- calibrated verification decisions;
- automatic story progression from vision;
- public installer hardening beyond development distribution;
- game injection, memory inspection, input automation, or in-game overlay.

8.5 B-2 work package 1: Native Companion process architecture

8.5.1 Recommended process model

Desktop/Web UI

```
|
| versioned local protocol
v
```

Companion Coordinator

- +-- Capture Service
- +-- Hotkey Service
- +-- Frame Quality Service
- +-- Recording Session Manager
- +-- Vision Worker Adapter placeholder
- +-- Diagnostics and Health Service

The Companion Coordinator **SHOULD** be the only process directly exposed to the frontend. Capture internals and future vision workers remain behind narrow internal APIs.

8.5.2 Process-isolation requirements

- a capture crash **SHOULD** not corrupt Studio data;
- a future model-worker crash **SHOULD** not terminate the UI;
- process restart **MUST** invalidate unsafe in-flight sessions;
- session ownership **MUST** be explicit;
- shutdown **MUST** release capture resources and clear transient frames;
- development logs **MUST** avoid raw frame serialization unless diagnostic retention is enabled.

8.6 B-2 work package 2: Capture-target selection

8.6.1 Required behavior

The user must explicitly select the application window or approved display target. The UI **SHALL** show:

- application/window title;
- live thumbnail where permitted;
- current dimensions;
- capture status;

- whether the source is minimized, occluded, closed, resized, or unavailable;
- a clear Change Window action.

The Companion SHOULD recognize likely Sea of Thieves windows for convenience, but MUST NOT silently select one without user confirmation.

8.6.2 Source-change behavior

If the game window:

- resizes - renegotiate buffers and continue where possible;
- changes resolution - record the new normalization metadata;
- is minimized - pause or fail with a clear capture-unavailable state;
- closes - stop capture and clear the transient buffer;
- is replaced by a new process/window - require confirmation or an approved safe reconnection rule;
- enters a capture-incompatible fullscreen mode - explain the recommended display mode.

8.7 B-2 work package 3: Creator recording mode

Creator recording sessions SHALL support categories:

- target reference;
- accepted-area walk;
- boundary;
- environmental variation;
- nearby hard negative;
- distant hard negative;
- correct object from invalid pose;
- diagnostic positive;
- diagnostic negative.

8.7.1 Recording session metadata

Each recording must include:

- waypoint draft version ID;
- recording category;
- creator and device ID;
- capture-source identity;
- start and end timestamps;
- resolution and aspect ratio;
- estimated frame rate;
- field-of-view value if creator supplies it;
- graphics/environment notes if supplied;
- frame-quality summary;
- local artifact hash and size;
- upload or local-storage state;
- user-provided label and notes.

8.7.2 Creator controls

- Start Recording;

- Pause;
- Resume;
- Stop and Review;
- Discard;
- Retry;
- Mark the recording category;
- Add a note;
- confirm whether the recording may be uploaded for a cloud-assisted build.

The recording UI **MUST** prevent accidental assignment to the wrong waypoint draft.

8.8 B-2 work package 4: Player hold-to-scan mode

8.8.1 Interaction contract

The default interaction is a press-and-hold action, initially targeted at approximately five seconds. The duration remains waypoint-configurable within governed limits.

Hotkey down

- > initialize scan session
- > begin buffering and progress events
- > capture at configured rate
- > monitor quality and motion

Hotkey release or maximum duration

- > finalize capture
- > select best frames
- > clear unnecessary frames
- > emit scan evidence metadata

8.8.2 Required scan behavior

- very short accidental presses are canceled without error;
- the scan may finish automatically at the maximum duration;
- the UI displays real capture state, not a fake timer;
- the user can cancel;
- repeated scans obey a small configurable cooldown to prevent accidental spam;
- only one scan may own the selected capture source at a time;
- creator recording and player scan conflicts are handled explicitly;
- loss of the game window cancels the scan safely;
- the scan includes enough frame timing to support future sequence analysis.

8.9 B-2 work package 5: Global hotkey and input safety

8.9.1 Requirements

- default key is configurable;
- conflicts with common game controls must be detectable or documented;
- the key may be keyboard, mouse, or supported controller input through a future adapter;
- the Companion observes the input only to initiate its own scan;
- it **MUST NOT** synthesize or forward game input;
- hotkey state must be visible and disableable;

- accessibility options should support toggle-to-scan as an alternative to hold where appropriate, while preserving explicit user initiation.

8.10 B-2 work package 6: Ring buffer and frame lifecycle

8.10.1 Ring-buffer intent

Maintain a bounded in-memory collection sufficient for:

- the active scan window;
- a small pre-roll to avoid losing the first useful view;
- frame-quality selection;
- short recovery from capture jitter.

8.10.2 Frame lifecycle rules

Captured frame

- > transient memory buffer
- > normalized candidate
- > quality metadata
- > selected evidence frame or discarded
- > vision worker in later phases
- > erased after result unless retention consent exists

No continuous disk recording is permitted in Player mode by default.

8.10.3 Resource limits

The Companion MUST enforce:

- maximum frame count;
- maximum memory budget;
- maximum scan duration;
- downsampled analysis resolution;
- backpressure if processing falls behind;
- frame-dropping strategy that preserves temporal distribution rather than only the newest frames.

8.11 B-2 work package 7: Frame normalization

The capture layer SHOULD produce normalized evidence with metadata, not silently destructive transformations.

Operations may include:

- color-space conversion;
- alpha removal;
- resolution downscaling;
- letterbox or crop metadata;
- HUD-safe-region metadata;
- timestamp normalization;
- orientation confirmation;
- duplicate hashing;
- optional multiple scales for later analysis.

The original capture dimensions and every transformation **MUST** be recorded so later geometric calculations can map back correctly.

8.12 B-2 work package 8: Capture-quality service

Each frame **SHOULD** receive objective measurements for:

- sharpness and motion blur;
- brightness and clipping;
- contrast;
- near-duplicate similarity;
- percentage dominated by sky, water, or near-uniform regions where feasible;
- obstruction or UI/menu likelihood;
- estimated camera motion relative to adjacent frames;
- temporal spacing;
- corruption or incomplete-frame detection.

The scan summary **SHOULD** include:

```
{
  "capturedFrameCount": 52,
  "usableFrameCount": 12,
  "selectedFrameCount": 9,
  "averageSharpness": 0.83,
  "brightnessStatus": "acceptable",
  "motionStatus": "good_slow_scan",
  "duplicateRate": 0.31,
  "captureContinuity": 0.97
}
```

These values do not decide the location. They decide whether the evidence is suitable for later judgment.

8.13 B-2 work package 9: Best-frame selection

The phase **SHALL** implement deterministic best-frame selection using capture quality and temporal diversity. Selected frames should:

- avoid severe blur;
- cover the full scan time;
- include different viewpoints;
- avoid near duplicates;
- preserve at least several connected adjacent frames for temporal analysis;
- remain within a bounded count appropriate for later inference.

The selection algorithm and its version **MUST** be recorded in scan metadata.

8.14 B-2 work package 10: Browser pairing and localhost security

8.14.1 Pairing flow

1. user launches the Companion;
2. Companion listens only on approved loopback interfaces;

3. website requests pairing;
4. Companion displays or receives a short-lived code;
5. user explicitly approves the requesting origin and account;
6. both sides derive or exchange a session secret;
7. capability negotiation occurs;
8. session expires after inactivity, logout, revocation, or configured duration.

8.14.2 Security requirements

- localhost binding only by default;
- strict origin allowlist;
- authenticated session on every command;
- anti-replay request IDs;
- short-lived pairing codes;
- no raw frame endpoint exposed to arbitrary web content;
- CORS or equivalent policy **MUST** not use wildcard access;
- rate limits for commands;
- explicit revocation UI;
- protocol-version negotiation;
- secure logging that does not expose pairing secrets.

8.15 B-2 work package 11: Desktop-integrated mode

When the shared UI runs inside the Windows desktop application:

- the app **SHOULD** connect to the integrated Companion automatically through a private channel;
- the user still explicitly selects the capture target;
- browser-origin pairing is not required, but process authentication remains;
- capabilities appear through the same platform-adapter contract;
- UI behavior remains nearly identical to paired web mode;
- the desktop shell **MUST NOT** bypass privacy indicators simply because the service is bundled.

8.16 B-2 work package 12: Capability and hardware discovery

The Companion **SHALL** report:

- capture API availability;
- current operating system and supported version;
- GPU vendors and relevant execution capability placeholders;
- available system memory;
- CPU class and logical cores;
- selected capture-source dimensions;
- estimated capture throughput;
- local storage availability for creator recordings;
- whether cloud-assisted build is allowed by policy;
- whether the system is in web-paired or desktop-integrated mode.

No hardware score should imply verification accuracy before B-4 exists.

8.17 B-2 work package 13: Privacy and consent surface

The Companion **MUST** visibly communicate:

- Vision Active or Vision Paused;
- selected capture source;
- whether Player mode is transient;
- whether Creator mode is recording to disk;
- whether any artifact is scheduled for upload;
- how to stop capture;
- how to delete retained recordings;
- how to revoke paired websites.

Default rules:

- runtime scans are memory-only;
- creator recordings are retained because the creator explicitly initiated recording;
- diagnostic retention is opt-in;
- cloud upload is opt-in and separately described;
- capture stops when the Tall Tale ends, the user logs out, the app closes, the source disappears, or permission is revoked.

8.18 B-2 work package 14: Failure and recovery model

Required capture failure codes include:

CAPTURE_SOURCE_NOT_SELECTED
 CAPTURE_SOURCE_CLOSED
 CAPTURE_SOURCE_MINIMIZED
 CAPTURE_SOURCE_UNAVAILABLE
 CAPTURE_PERMISSION_DENIED
 CAPTURE_FORMAT_CHANGED
 CAPTURE_FRAME_TIMEOUT
 CAPTURE_INSUFFICIENT_FRAMES
 CAPTURE_EXCESSIVE_BLUR
 CAPTURE_EXCESSIVE_DUPLICATES
 CAPTURE_EXCESSIVE_MOTION
 CAPTURE_INSUFFICIENT_MOTION
 CAPTURE_STORAGE_UNAVAILABLE
 PAIRING_REQUIRED
 PAIRING_EXPIRED
 PROTOCOL_INCOMPATIBLE
 COMPANION_UNAVAILABLE
 HOTKEY_CONFLICT

Each failure **MUST** map to:

- developer diagnostic detail;
- user-facing title;
- safe corrective action;
- retry eligibility;
- whether the current story state remains unchanged.

8.19 B-2 work package 15: Observability and diagnostic bundle

The Companion SHOULD produce a diagnostic bundle containing metadata only by default:

- application and protocol versions;
- hardware capability summary;
- capture-source metadata;
- session timing;
- frame counts and quality summaries;
- error codes;
- logs with secrets redacted;
- configuration relevant to the failure.

Raw frames may be added only through explicit user consent and SHOULD have an automatic expiration.

8.20 B-2 testing strategy

8.20.1 Automated tests

- protocol authentication and expiration;
- invalid-origin rejection;
- capture-session state machine;
- ring-buffer limits;
- frame-selection determinism;
- resolution-change handling;
- source-close cleanup;
- hotkey cancellation and cooldown;
- privacy default assertions;
- no disk writes in default Player mode;
- duplicate request idempotency;
- desktop and web adapter parity.

8.20.2 Manual test matrix

- game window at 1080p and 1440p;
- borderless window and normal window;
- resize during capture;
- minimize and restore;
- close and reopen;
- move between monitors;
- high and low frame rate;
- rapid camera motion;
- nearly stationary camera;
- dark scene;
- bright scene;
- menu visible;
- temporary Companion restart;
- browser pairing expiration;
- desktop integrated mode.

8.21 B-2 required demonstration

The demonstration SHALL show:

1. Companion launches and displays privacy state;
2. user selects the real Sea of Thieves window;
3. browser pairs through a short-lived secure session;
4. Studio records and previews a creator target recording;
5. Player initiates a five-second hold-to-scan;
6. the UI receives live scan progress;
7. the Companion returns a structured quality summary and selected-frame metadata;
8. no Player frames remain on disk after completion;
9. resizing or temporarily losing the window produces safe recovery behavior;
10. the same scan can be initiated in desktop-integrated mode;
11. a mock verification result continues through the B-1 story path.

8.22 B-2 exit gate

Phase B-2 Exit Gate

Phase B-2 is complete when capture, recording, scan initiation, frame lifecycle, pairing, privacy, quality measurement, failure recovery, and adapter integration are real and tested. The result may still use a mock location decision, but it must be based on a genuine structured scan produced by the selected game window.

8.22.1 Mandatory acceptance checklist

- ☐ Sea of Thieves window can be explicitly selected.
- ☐ Creator recordings are categorized and persisted safely.
- ☐ Player scans use a bounded transient ring buffer.
- ☐ Player frames are not written to disk by default.
- ☐ Hold-to-scan progress reflects real capture state.
- ☐ Frame quality and best-frame selection are implemented.
- ☐ Browser pairing is authenticated, origin-restricted, expiring, and revocable.
- ☐ Desktop mode uses the same capability contract.
- ☐ Capture stops cleanly on logout, close, source loss, and revocation.
- ☐ Resolution changes and source failure produce clear states.
- ☐ Diagnostic metadata is useful without exposing frames.
- ☐ Tests and real-window demonstration pass.

8.23 B-2 principal risks and mitigations

Risk	Consequence	Required mitigation
Exclusive fullscreen or capture incompatibility	Frozen or black scans	Detect health failure; recommend supported borderless/windowed mode
Localhost service exposed to arbitrary websites	Privacy breach	Loopback-only, origin approval, authenticated session, no wildcard policy

Risk	Consequence	Required mitigation
Capture consumes excessive GPU resources	Game performance degrades	Bounded capture rate, downscaled analysis, profiling, backpressure
Frames accidentally retained	Privacy violation	Memory-only Player mode, automated disk-write tests, cleanup on crash/restart
UI reports scanning while capture is dead	Interface truth violation	Health heartbeat and real progress from Companion state
Hotkey conflicts with game control	Bad player experience	Configurable binding, conflict warning, cancellation, accessibility alternative

8.24 B-2 Codex completion evidence

Codex MUST provide:

- native-process diagram and command allowlist;
- pairing threat model;
- capture performance report;
- Player frame-lifecycle proof;
- supported display-mode notes;
- automated-test results;
- browser and desktop demonstration recording or screenshots;
- failure-code catalog;
- list of all disk locations used by creator recordings and diagnostics;
- cleanup and rollback instructions.

Phase B-3: Vision Waypoint Studio Authoring

9.1 Phase mission

Build the complete guided Studio experience that allows a nontechnical creator to gather, organize, validate, resume, and submit every piece of evidence required to create a high-quality Vision Waypoint. Codex shall build the tool once; creators shall use the tool repeatedly without command lines, notebooks, JSON editing, or custom code.

At completion, a creator can produce a valid build-input package for a waypoint entirely through the Studio and Companion. The actual build may still use a transitional or partial backend until B-4, but the authoring data, state, validation, and artifact handoff **MUST** be real.

9.2 Creator experience objective

The creator should feel that they are placing and testing a checkpoint in a game editor. The interface may explain why diverse views and similar wrong locations matter, but it must not require the creator to understand embeddings, feature descriptors, homographies, pose covariance, or neural-network architecture.

The system should translate technical need into direct guidance:

- not “insufficient baseline diversity,” but “capture the right side of the landmark”;
- not “weak negative separation,” but “record the similar rock nearby”;
- not “pose coverage gap,” but “step farther away and record the approach”;
- not “feature density too low,” but “include more of the surrounding cliff and tree.”

9.3 In scope

- Vision Waypoint Library;
- creation wizard;
- waypoint-type selection;
- Companion connection and capture health;
- target recording;
- coverage guidance;
- accepted-area recording and editor;
- boundary and excluded-area definition;
- hard-negative recording manager;
- stable-region suggestions and editor;
- recording review, trimming, labeling, and deletion;
- resumable drafts;
- data-health validation;

- build-input package generation;
- build status and placeholder/partial processing display;
- positive and negative test workflow foundation;
- Guided, Detailed, and Engineering creator modes;
- accessibility and responsive layout;
- waypoint usage and version information.

9.4 Out of scope

- final calibrated vision engine;
- automatic reliability grade based on the complete B-4 pipeline;
- automatic story progression from real results;
- unrestricted public marketplace;
- fully live AR authoring.

9.5 B-3 work package 1: Vision Waypoint Library

9.5.1 Library placement

The Studio navigation SHALL include a first-class Vision Waypoint area beside other reusable assets. A recommended structure is:

```
Studio
  Tall Tales
  Assets
  Characters
  Locations
  Vision Waypoints
  Publishing
```

9.5.2 Waypoint card information

Each waypoint card SHOULD show:

- thumbnail;
- waypoint name;
- type;
- island or region;
- current draft state;
- highest published version;
- reliability grade if available;
- last tested date;
- compatibility status;
- number of Tall Tales using it;
- creator or owner;
- privacy/share scope.

9.5.3 Library actions

Creators with permission may:

- create;

- open;
- duplicate;
- rename;
- archive;
- create a new version;
- compare versions;
- export;
- import;
- retest;
- view usage;
- deprecate a published version;
- choose an existing published version for a story.

The Library **MUST** prevent deletion or mutation that would invalidate active or historical stories.

9.5.4 Filters and search

Support filters for:

- type;
- island/region;
- authoring state;
- reliability grade;
- used/unused;
- private/crew/community/official scope;
- compatible/outdated;
- recently modified.

9.6 B-3 work package 2: New-waypoint purpose selection

The wizard begins with the story question:

What must the player prove?

Supported initial types:

9.6.1 Area Arrival

The player has reached a general area. Recognition emphasizes contextual scene localization and allows a larger pose region.

9.6.2 Exact Landmark

The player has found a specific fixed object or structure such as a rock, statue, doorway, painting, shrine, shipwreck, or cave entrance.

9.6.3 Viewpoint

The player must stand in an approved area and face the correct composition or alignment.

9.6.4 Object Inspection

A specific object must be clearly visible at an appropriate screen size and within the correct scene context.

9.6.5 Item Pickup

The player must first verify the location, then present or hold an expected item or proxy object.

9.6.6 Sequence

Several Vision Waypoints or inspections must be completed in a required order.

9.6.7 Advanced

Allows an experienced creator to combine rules while remaining inside governed package and verification contracts.

9.6.8 Type-specific guidance

The wizard **MUST** adapt instructions and required data to the selected type. It must not ask a generic beach creator to capture the same evidence as a planar carved wall without explaining the difference.

9.7 B-3 work package 3: Story-purpose and completion actions

The waypoint proves a visual condition. Story behavior remains a separate event configuration.

The wizard should ask:

- What does successful verification mean in the story?
- Should this waypoint be reusable by other stories?
- Is this story-critical?
- What player guidance is allowed?
- Is Captain fallback permitted?

Possible later success actions are displayed for context:

- advance objective;
- reveal image or video;
- run a Three.js scene;
- reveal text or journal entry;
- play audio;
- notify Captain;
- unlock another block;
- emit custom governed event.

B-3 stores the intended success event configuration but does not hard-code the cinematic into the waypoint package.

9.8 B-3 work package 4: Companion connection step

The wizard **SHALL** show:

- connected device name;
- desktop-integrated or web-paired mode;
- selected capture window;
- current resolution;
- capture health;
- available local storage;
- hardware capabilities;
- privacy mode;

- local or cloud-assisted build preference;
- clear repair actions if unavailable.

The creator may save the draft without the Companion and resume later. Recording steps remain blocked with a truthful explanation until capture is available.

9.9 B-3 work package 5: Target recording workflow

9.9.1 Creator instruction

The interface should guide the creator to record the landmark from multiple positions while maintaining overlap:

1. begin near a useful front view;
2. move slowly left and right;
3. capture close, medium, and farther distances;
4. include stable surroundings;
5. vary camera pitch moderately;
6. avoid very fast turns;
7. capture environmental variations where practical.

9.9.2 Live guidance

The Studio SHALL consume B-2 metadata and provide live, truthful messages such as:

- “Move more slowly.”
- “This angle is already well covered.”
- “Capture more of the left side.”
- “Step farther back.”
- “Keep the landmark and surrounding cliff visible.”
- “The image is too dark to evaluate.”
- “Enough front coverage has been recorded.”

B-3 may use provisional image-diversity and motion heuristics before B-4. It MUST clearly distinguish provisional coverage from final reliability.

9.9.3 Coverage categories

The target step SHOULD display:

- front;
- front-left;
- left;
- front-right;
- right;
- close;
- medium;
- far;
- upward pitch;
- downward pitch;
- day/bright;
- night/dark;
- weather variation.

Not every waypoint requires every category. Requirements are type-specific and configurable by the guided profile.

9.10 B-3 work package 6: Recording review and curation

After capture, the creator can:

- preview video;
- inspect automatically selected representative frames;
- trim unusable beginning or ending segments;
- split one recording into categories;
- merge metadata across compatible recordings;
- relabel;
- add notes;
- delete;
- replace;
- mark environmental conditions;
- flag a section that should not be used.

The system SHOULD automatically identify:

- severe blur;
- long duplicate stretches;
- menus and loading screens;
- source interruptions;
- likely unrelated footage;
- missing metadata.

Automatic removal MUST be reviewable or at least reported. The creator should not wonder why half the recording vanished into the machine's private ocean.

9.11 B-3 work package 7: Accepted player region

9.11.1 Guided walk mode

The creator physically walks through positions where a player should pass verification. The Companion records the sequence and the Studio later maps it to an accepted pose region.

The guided instructions should include:

- stand at the closest acceptable point;
- move through normal approach positions;
- stand at the farthest acceptable point;
- face the target from several positions;
- record any separate valid approach zone;
- avoid entering positions that should not pass.

9.11.2 Detailed editor mode

The Studio SHOULD display a simplified top-down or three-dimensional representation when reconstruction data becomes available. Before B-4, a provisional region editor may use creator-defined distance and directional rules.

Editable rules include:

- minimum distance;
- maximum distance;
- accepted side or approach;
- excluded zones;
- minimum target visibility;
- maximum facing angle;
- minimum contextual coverage;
- multiple disconnected accepted regions;
- whether standing behind the object is forbidden;
- whether movement through an area is sufficient or a stable view is required.

9.11.3 Rule presentation

The creator sees plain language:

Player must be within approximately 2 to 7 meters.

Player must be in front of the landmark.

Player must face within 55 degrees of the landmark center.

At least half of the landmark must be visible.

Engineering mode may show coordinate volumes and pose distributions.

9.12 B-3 work package 8: Boundary and invalid-pose recording

The creator SHALL be asked to record the edges of the valid region and important invalid conditions:

- just too far away;
- just behind the object;
- correct object from forbidden side;
- correct area while facing away;
- correct island but outside the target zone;
- target visible only as a tiny distant feature.

These recordings are critical because accuracy depends on distinguishing a valid checkpoint from nearby almost-valid conditions.

9.13 B-3 work package 9: Hard-negative recording workflow

9.13.1 Required creator task

For high-reliability waypoints, recording similar wrong places is mandatory.

The UI explains:

Record nearby places or objects that a player or computer might mistake for the target. The strongest waypoint is not merely familiar with the right location; it is familiar with its most convincing impostors.

9.13.2 Negative categories

- nearby similar rock or structure;
- opposite side of target;
- similar cave entrance;
- repeated game prop;
- wrong beach on the correct island;

- similar distant island location;
- correct object outside accepted region;
- correct general scene but wrong focal landmark;
- creator-discovered near false positive;
- field-diagnostic near false positive.

9.13.3 Negative requirements by profile

Profile	Minimum expectation before final calibration
Balanced	At least one meaningful nearby negative when confusers exist
Strict	Several nearby and boundary negatives
Story-Critical	Required nearby, boundary, invalid-pose, and strongest discovered confusers
Custom	Creator settings may increase but not bypass governing safety minima

B-4 determines exact sufficiency from the data. B-3 prevents the creator from skipping the entire step.

9.14 B-3 work package 10: Stable and ignored region editor

9.14.1 Purpose

Mark fixed visual evidence and de-emphasize unreliable regions.

9.14.2 Suggested important regions

- rock cracks;
- cliff edges;
- building beams;
- door frames;
- stone seams;
- statue contours;
- tree trunks;
- fixed architectural relationships;
- persistent terrain silhouettes.

9.14.3 Suggested ignored regions

- water;
- sky and clouds;
- grass and moving foliage;
- smoke, fire, rain, and particles;
- players, skeletons, animals, and ships;
- held objects and hands unless explicitly required;
- HUD elements;
- temporary loot;
- highly reflective or animated surfaces.

9.14.4 Editor behavior

The creator can use:

- Important brush;
- Ignore brush;
- Eraser;
- accept suggestion;
- reset suggestion;
- copy region intent to related frames;
- inspect multiple reference viewpoints;
- preview the aggregate mask.

A single two-dimensional mask is insufficient for a multi-view location. The package must support per-frame or projected region definitions.

9.15 B-3 work package 11: Data-health dashboard

Before build submission, the Studio SHALL summarize:

- target coverage;
- accepted-area coverage;
- boundary evidence;
- number and diversity of hard negatives;
- capture-quality problems;
- environmental variation;
- stable-region confirmation;
- missing required metadata;
- unresolved source interruptions;
- expected verification profile.

Possible statuses:

READY_TO_BUILD

BUILDABLE_WITH_WARNINGS

NOT_READY

Warnings must be actionable.

Example:

“The target is well captured from the front, but the right-side approach has no usable footage. Record one slow pass from the right before Story-Critical calibration.”

9.16 B-3 work package 12: Resumable wizard and draft safety

Every step MUST save progress safely. The creator may:

- leave Studio;
- switch devices;
- reconnect the Companion;
- continue after a failed upload;
- replace one recording without restarting;
- create notes for collaborators;

- review an audit history.

The wizard SHALL show:

- completed steps;
- incomplete steps;
- blocked steps;
- warnings;
- last saved time;
- who last modified the draft;
- recommended next action.

Concurrent editing SHOULD use locking, optimistic concurrency, or a collaboration model that prevents silent overwrite.

9.17 B-3 work package 13: Build-input package

When the draft is ready, Studio creates a versioned build-input package or manifest referencing retained artifacts.

Required contents include:

- waypoint and draft-version identity;
- waypoint type;
- verification profile;
- recording inventory and categories;
- artifact hashes and transformation metadata;
- accepted and excluded region definitions;
- stable and ignored region data;
- creator-provided FOV/environment metadata;
- requested local or cloud build mode;
- creator guidance preferences;
- Captain fallback setting;
- schema version.

The build-input package MUST be deterministic and independently validateable.

9.18 B-3 work package 14: Test workflow foundation

B-3 implements the interface and persistence for guided tests even though B-4 supplies final verification.

Test categories:

- standard positive approach;
- alternate valid approach;
- close positive;
- far positive;
- lighting/environment positive;
- nearby hard negative;
- boundary negative;
- facing-away negative;
- distant confuser;
- creator-defined custom case.

Each test stores:

- expected result;
- actual result when available;
- recording/attempt reference;
- creator confirmation;
- notes;
- whether the case should enter calibration or remain a locked test.

9.19 B-3 work package 15: Creator modes

9.19.1 Guided mode

- plain-language steps;
- automatic defaults;
- required data collection;
- no raw thresholds;
- limited advanced terminology;
- recommended for most creators.

9.19.2 Detailed mode

- accepted-region editor;
- target masks;
- category management;
- test-set management;
- guidance tuning;
- verification-profile controls.

9.19.3 Engineering mode

- frame and feature diagnostics;
- reconstruction preview when available;
- threshold and distribution inspection;
- package contents;
- model and engine versions;
- inlier and pose visualization in later phases.

Mode changes MUST not create incompatible waypoint data. They change the amount of control shown, not the underlying package standard.

9.20 B-3 work package 16: Accessibility and presentation quality

The wizard MUST support:

- keyboard navigation;
- visible focus;
- screen-reader labels;
- text alternatives for status conveyed by color;
- reduced-motion option while retaining progress truth;
- readable contrast;
- captions or text for instructional media;
- scalable UI at common laptop resolutions;

- non-hover access to critical information;
- clear confirmation before destructive recording deletion.

The interface should remain cinematic and polished, but authoring clarity takes precedence over theatrical fog drifting across required controls.

9.21 B-3 testing strategy

9.21.1 Automated

- wizard state persistence;
- required-step validation;
- recording-category rules;
- immutable published-version behavior;
- concurrent-edit conflict handling;
- package-schema validation;
- library filtering and permissions;
- accessibility checks;
- web/desktop parity;
- error recovery after Companion disconnect.

9.21.2 Human usability

Test with at least:

- the project owner using Guided mode;
- a technically comfortable but uninvolved user;
- a user with little computer-vision or development knowledge.

Observe whether users can:

- understand what to record;
- distinguish target and negative recordings;
- resume a draft;
- fix a warning;
- determine when the waypoint is ready;
- attach notes and review recordings;
- complete the build-input handoff without developer help.

9.22 B-3 required demonstration

The demonstration SHALL show:

1. create a new Exact Landmark waypoint;
2. pair or use the integrated Companion;
3. record target coverage from several angles;
4. review and trim a recording;
5. record an accepted-area walk;
6. record at least two similar wrong locations;
7. edit important and ignored regions;
8. leave the wizard and resume later;
9. resolve an actionable data-health warning;

10. generate a schema-valid build-input package;
11. view the waypoint in the Library with draft status and usage metadata;
12. perform the same authoring workflow in web-paired and desktop-integrated modes where practical.

9.23 B-3 exit gate

Phase B-3 Exit Gate

Phase B-3 is complete when a creator can produce a complete, persistent, versioned, schema-valid Vision Waypoint build-input package entirely through the Studio and Companion. The experience must be understandable without terminal use or model knowledge, and it must preserve all data required by the governing verification architecture.

9.23.1 Mandatory acceptance checklist

- ☐ Vision Waypoint Library is first-class and reusable.
- ☐ All initial waypoint types are represented.
- ☐ Guided creator wizard is complete and resumable.
- ☐ Target, accepted-area, boundary, environmental, and negative recordings are supported.
- ☐ Coverage and quality guidance are truthful.
- ☐ Stable/ignored regions can be reviewed across viewpoints.
- ☐ Data-health warnings explain specific corrective actions.
- ☐ Build-input package validates against versioned schema.
- ☐ Draft edits cannot mutate published versions.
- ☐ Web and desktop modes share the workflow.
- ☐ Accessibility and usability testing pass.
- ☐ No command line, custom JSON, or Codex prompt is required to create the input package.

9.24 B-3 principal risks and mitigations

Risk	Consequence	Required mitigation
Wizard collects attractive but insufficient footage	B-4 cannot build accurate waypoint	Type-specific requirements, live quality guidance, data-health gate
Nontechnical language hides important requirements too aggressively	Creator cannot fix reliability problems	Layered disclosure: Guided explanation plus Detailed/Engineering evidence
Accepted-region editor promises geometry before reconstruction exists	False precision	Label provisional data clearly; finalize against B-4 reconstruction
Creator skips hard negatives	False-positive risk	Profile-specific mandatory negative step and readiness gate
Long wizard becomes exhausting	Abandoned drafts	Resumability, progress, step independence, recommended next action
Desktop and web workflows diverge	Double maintenance	Shared UI and platform-adaptor contract from B-1

9.25 B-3 Codex completion evidence

Codex MUST provide:

- complete wizard route map;
- screenshots of each step and all states;
- build-input schema and sample package;
- accessibility test results;
- usability test notes and discovered changes;
- recording and draft persistence tests;
- web/desktop parity evidence;
- data-health rule catalog;
- demonstration recording;
- rollback and migration notes.

Phase B-4: Verification Build and Runtime Engine

10.1 Phase mission

Implement the complete, modular computer-vision pipeline that converts Studio-authored recordings into calibrated, versioned runtime packages and evaluates player scans through several independent mandatory gates. This phase must provide high viewpoint tolerance without sacrificing false-positive protection.

At completion, at least three pilot Vision Waypoints will be built from Studio data and evaluated in shadow mode against unseen valid approaches, boundaries, and dangerous hard negatives. No real scan will advance the story automatically until B-5.

10.2 Core engineering rule

No Single-Score Approval

The verification engine **MUST NOT** approve a waypoint from one screenshot, one model, one nearest-neighbor result, or one blended confidence score. Every required gate must pass. Strong evidence in one gate cannot compensate for a failed geometric, pose, negative-separation, temporal, or checkpoint-specific gate.

10.3 In scope

- deterministic build pipeline;
- frame preparation and dataset partitioning;
- global visual place retrieval;
- hard-negative indexing and veto;
- local feature extraction and matching;
- geometric verification;
- sparse 3D reconstruction;
- camera pose localization;
- accepted pose-volume finalization;
- stable-region and spatial-coverage rules;
- temporal sequence consistency;
- checkpoint-specific rule modules;
- calibration and reliability reports;
- runtime package generation;
- model and engine adapters;
- local runtime integration through the Companion;
- pilot waypoint test program;

- performance profiling and fallback paths;
- reproducibility and package compatibility.

10.4 Out of scope

- automatic progression in production stories;
- continuous background scanning;
- live AR compositing;
- unrestricted public sharing;
- custom model training unless justified by pilot evidence;
- universal recognition of every game location.

10.5 B-4 work package 1: Engine modularity and replaceable model adapters

10.5.1 Architectural objective

The waypoint format, Studio workflow, and story engine **MUST** not depend on a particular research library or neural model. The build and runtime engine should use stable interfaces such as:

```
class GlobalRetriever:
    def build_index(self, references): ...
    def query(self, frames, top_k): ...

class LocalMatcher:
    def extract(self, frames, masks=None): ...
    def match(self, query, reference): ...

class ReconstructionEngine:
    def build(self, references, matches): ...

class PoseLocalizer:
    def localize(self, query_features, map_data): ...

class CalibrationEngine:
    def calibrate(self, positives, negatives, profile): ...
```

Candidate implementations may initially include DINOv2-derived place descriptors, SALAD or AnyLoc-style retrieval, SuperPoint and LightGlue, LoFTR or EfficientLoFTR, COLMAP or PyCOLMAP, OpenCV geometry, and ONNX Runtime deployment. These are replaceable modules, not schema identities.

10.5.2 Required metadata

Every build and attempt **MUST** record:

- engine semantic version;
- model bundle versions;
- feature extractor version;
- matcher version;
- reconstruction version;
- calibration algorithm version;
- frame-selection version;
- hardware execution provider;

- package schema version.

10.6 B-4 work package 2: Build-job orchestration

10.6.1 Build states

QUEUED
 VALIDATING_INPUT
 PREPARING_FRAMES
 EXTRACTING_GLOBAL_FEATURES
 EXTRACTING_LOCAL_FEATURES
 BUILDING_RECONSTRUCTION
 INDEXING_NEGATIVES
 CALIBRATING
 PACKAGING
 VALIDATING_PACKAGE
 COMPLETED
 FAILED
 CANCELED
 NEEDS_ADDITIONAL_DATA

10.6.2 Build requirements

- builds are resumable where practical;
- completed intermediate artifacts are content-addressed and reusable;
- retries do not duplicate immutable artifacts;
- canceling a build leaves the draft recoverable;
- progress events are truthful and map to actual work;
- failures identify the stage and corrective action;
- local and cloud-assisted builds produce compatible package formats;
- package output is deterministic for equivalent inputs and engine versions where practical.

10.7 B-4 work package 3: Dataset preparation

10.7.1 Frame extraction

For creator recordings:

- extract candidate frames at a controlled interval;
- retain timestamps and camera-sequence relationships;
- apply B-2 quality metrics;
- reject corruption, severe blur, menus, and long duplicate runs;
- preserve enough sequential frames for reconstruction;
- create multiple image scales as required;
- map stable/ignored regions to each frame;
- store transformations from source pixels to analysis pixels.

10.7.2 Dataset categories

The build **MUST** preserve categories:

- positive reference;

- accepted-region positive;
- boundary;
- environmental positive;
- hard negative;
- invalid pose;
- calibration-only;
- locked test.

Calibration data and locked test data **MUST** remain separate. The system must not tune thresholds against the same evidence used for final reported performance.

10.7.3 Duplicate and diversity handling

Near-duplicate frames should be reduced without erasing:

- angle coverage;
- distance coverage;
- environmental variation;
- temporal chains needed for reconstruction;
- rare but valid viewpoints.

The build report **SHALL** explain how many frames were extracted, rejected, clustered, retained, and reserved.

10.8 B-4 work package 4: Global visual place retrieval

10.8.1 Purpose

Quickly identify which target reference views are most visually related to each query frame and measure whether known wrong places are similarly convincing.

10.8.2 Requirements

For each selected scan frame:

1. create one or more global descriptors;
2. query the target reference index;
3. query the hard-negative index;
4. return top target candidates and top negative candidates;
5. calculate target quality and target-versus-negative separation;
6. aggregate evidence across frames without allowing one exceptional frame to hide widespread disagreement.

10.8.3 Multi-region retrieval

The engine **SHOULD** support descriptors for:

- full frame;
- central region;
- target-probable region;
- left and right context;
- scale variants;
- creator-marked important regions.

This helps when the landmark occupies only part of the view.

10.8.4 Retrieval gate

The gate requires both:

`best_target_similarity >= waypoint_target_minimum`

AND

`best_target_similarity - best_negative_similarity >= waypoint_negative_margin`

The actual metric need not be normalized to 0-1, but the calibration report must make the threshold meaningful.

10.8.5 Unknown-scene rejection

A scan that resembles neither target nor known negatives is not automatically safe. It is `NOT_AT_TARGET` or `INSUFFICIENT_VISUAL_EVIDENCE` depending on capture quality and context.

10.9 B-4 work package 5: Hard-negative system

10.9.1 Negative index

The runtime package SHALL contain efficient representations of:

- nearby confusers;
- boundary views;
- invalid poses;
- repeated game props;
- creator-specified distant confusers;
- mined near false positives.

10.9.2 Veto behavior

A negative may veto verification through:

- global similarity too close to target;
- stronger local geometry than target;
- successful localization into a negative scene map;
- target object detected but pose outside accepted region;
- repeated asset match without required contextual landmarks.

10.9.3 Hard-negative mining

The build and field-test tools SHOULD rank rejected negatives by closeness to acceptance. The creator can promote near misses into the next draft's hard-negative set.

10.9.4 Safety rule

Any known negative accepted during calibration or locked testing marks the build `UNSAFE`. It may not be published for automatic progression.

10.10 B-4 work package 6: Local feature extraction

10.10.1 Purpose

Find stable visual points or semi-dense correspondences that represent actual scene structure rather than broad visual appearance.

10.10.2 Requirements

- respect creator important/ignored regions;
- de-emphasize water, sky, foliage, HUD, hands, and transient entities;
- retain descriptor provenance;
- support multiple scales;
- cache reference features at build time;
- extract only query features at runtime;
- report feature density and distribution;
- support at least a primary and secondary matcher path.

10.10.3 Primary and secondary methods

A recommended initial strategy is:

- primary: sparse keypoint detector/descriptor plus LightGlue-style matching;
- secondary: detector-free or semi-dense matcher for low-texture or difficult views.

The secondary method may support, contradict, or rescue the primary result according to calibrated rules. It must not become an unconditional approval path.

10.11 B-4 work package 7: Local feature matching

For each promising target candidate:

- match query features to reference features;
- apply mutual or confidence filtering as appropriate;
- remove matches in ignored regions;
- preserve match coordinates and confidences;
- associate reference features with 3D map points when available;
- calculate raw matches, filtered matches, and spatial distribution;
- compare equivalent local matching against dangerous negatives.

A large raw match count is not proof. Repeated textures can produce many misleading matches.

10.12 B-4 work package 8: Geometric verification

10.12.1 Planar or approximately planar target

Use robust homography or equivalent projective verification for:

- walls;
- carved rock faces;
- tablets;
- signs;
- paintings;
- flat architectural surfaces.

Required measurements SHOULD include:

- number of inliers;
- inlier ratio;
- reprojection error;

- transformation scale and skew;
- perspective plausibility;
- match coverage across the target;
- agreement with contextual landmarks.

10.12.2 Full 3D location

Use essential/fundamental geometry and query-to-map correspondences for:

- beaches;
- caves;
- ruins;
- cliff paths;
- outposts;
- multi-object natural scenes.

10.12.3 Robust estimation

Outlier rejection **MUST** be used. Geometry that implies an impossible or unstable camera relation fails regardless of descriptor similarity.

10.13 B-4 work package 9: Sparse 3D reconstruction

10.13.1 Build objective

Use overlapping positive reference views to construct a local coordinate system and sparse map containing:

- camera pose for reference frames;
- stable 3D feature points;
- observations linking images to points;
- descriptors or feature associations;
- target landmark location or target surface;
- optional scale approximation where known;
- reconstruction quality metrics.

10.13.2 Reconstruction validation

The build **MUST** check:

- enough connected images;
- sufficient baseline diversity;
- low reprojection error;
- absence of disconnected or contradictory components;
- coverage of accepted player regions;
- coverage of the target landmark;
- stable important-region participation.

If reconstruction is weak, the build returns `NEEDS_ADDITIONAL_DATA` with creator-facing guidance such as:

“The right-side views do not overlap enough with the front views. Record one slow pass connecting those angles.”

10.13.3 Multiple maps

A waypoint MAY contain:

- one target map;
- several target submaps for disconnected accepted approaches;
- dedicated negative maps for dangerous confusers;
- a planar target model plus contextual 3D map.

10.14 B-4 work package 10: Camera pose localization

10.14.1 Runtime process

For each usable query frame:

1. retrieve likely reference views;
2. match local features;
3. associate matches with 3D points;
4. estimate camera position and orientation robustly;
5. refine pose where appropriate;
6. calculate uncertainty and reprojection error;
7. determine whether the pose lies inside an accepted region;
8. determine whether the camera faces the required target zone.

10.14.2 Pose outputs

position: x, y, z
 orientation: yaw, pitch, roll or quaternion
 inlier count
 inlier ratio
 reprojection error
 pose uncertainty
 localized map/submap
 accepted-region membership
 facing-angle result
 target visibility estimate

10.14.3 Viewpoint tolerance

The player need not stand on a recorded camera pose. The pose is estimated within the reconstructed scene, allowing positions between and around reference viewpoints as long as evidence remains strong and the pose lies in the accepted region.

10.15 B-4 work package 11: Accepted pose-region finalization

B-3 creator data becomes a concrete runtime rule after reconstruction.

Supported region forms SHOULD include:

- sphere or distance band;
- cylinder;
- axis-aligned or oriented box;
- polygon or polyhedron;

- union of several accepted volumes;
- excluded sub-volumes;
- target-facing cone;
- minimum and maximum target screen coverage.

The build **SHOULD** visualize how creator recordings map into the finalized region and warn if the creator intended a region not supported by reconstruction coverage.

10.16 B-4 work package 12: Spatial evidence coverage

10.16.1 Why raw count is insufficient

One repeated texture patch may generate many matches. Verification requires evidence distributed across meaningful regions.

10.16.2 Coverage methods

The engine **MAY** combine:

- grid-cell coverage;
- convex hull area;
- creator-region coverage;
- target plus contextual-region requirements;
- independent landmark groups;
- depth or 3D-point spread.

Example requirement for an exact rock:

- upper crack region confirmed;
- lower edge confirmed;
- adjacent fixed tree or cliff confirmed;
- matches distributed across at least a defined portion of the visible target.

A match cluster confined to one tiny patch fails the coverage gate.

10.17 B-4 work package 13: Temporal and sequence consistency

10.17.1 Multi-frame requirement

A scan result **SHALL** use several frames distributed across the scan. A typical runtime may retain 8-15 frames, though calibration determines exact use.

10.17.2 Consistency checks

- enough frames localize successfully;
- position estimates remain physically consistent;
- camera orientation changes smoothly;
- no impossible jumps between maps;
- target identity remains consistent;
- geometric quality remains above minimum;
- retrieval does not alternate between target and negative scenes;
- optical flow or adjacent-frame feature motion is coherent where used.

10.17.3 Example physically plausible sequence

Frame 1: same position, yaw 0 degrees

Frame 2: same position, yaw 5 degrees

Frame 3: same position, yaw 11 degrees

Frame 4: slight step left, yaw 17 degrees

An implausible sequence that teleports around the reconstructed map fails even if isolated frames appear convincing.

10.18 B-4 work package 14: Checkpoint-specific verification modules

10.18.1 Exact Landmark

Require:

- target scene retrieval;
- target or surface geometry;
- camera inside approved region;
- target visibility;
- contextual landmark evidence;
- temporal agreement.

10.18.2 Area Arrival

Require:

- localization into approved area;
- several contextual landmarks;
- adequate scene coverage;
- reduced dependence on one small object;
- larger but still bounded pose region.

10.18.3 Viewpoint

Require:

- camera in the approved area;
- orientation within tolerance;
- required composition or target intersection;
- optional alignment of multiple landmarks;
- stable view for enough frames.

10.18.4 Object Inspection

Require:

- correct scene context;
- object match;
- object size consistent with distance;
- object occupies enough screen area;
- visibility not dominated by obstruction.

10.18.5 Item Pickup

Require two governed stages:

1. location verified;
2. expected proxy item visible in the held-item region for enough frames.

The item by itself cannot prove the location.

10.18.6 Sequence

Require child waypoint completions in order with session and story-state validation. A repeated completion event must not skip sequence steps.

10.19 B-4 work package 15: Outcome and failure model

Internal outcomes:

VERIFIED
 INSUFFICIENT_VISUAL_EVIDENCE
 NOT_AT_TARGET
 AMBIGUOUS
 SYSTEM_ERROR

10.19.1 Example failure reasons

QUALITY_TOO_FEW_USABLE_FRAMES
 QUALITY_EXCESSIVE_BLUR
 QUALITY_TARGET_TOO_SMALL
 RETRIEVAL_TARGET_TOO_WEAK
 RETRIEVAL_NEGATIVE_TOO_CLOSE
 LOCAL_MATCH_INSUFFICIENT
 GEOMETRY_INVALID
 POSE_NOT_LOCALIZED
 POSE_OUTSIDE_ACCEPTED_REGION
 POSE_WRONG_ORIENTATION
 TARGET_VISIBILITY_INSUFFICIENT
 SPATIAL_COVERAGE_INSUFFICIENT
 TEMPORAL_INCONSISTENT
 CHECKPOINT_RULE_FAILED
 MODEL_OR_PACKAGE_INCOMPATIBLE

Player guidance is mapped later. Technical detail is retained for Captain and Engineering diagnostics.

10.20 B-4 work package 16: Calibration engine

10.20.1 Per-waypoint calibration

No universal threshold is sufficient. Calibration SHALL choose or validate waypoint-specific requirements using:

- positive calibration attempts;
- boundary attempts;
- hard negatives;

- invalid poses;
- environmental variants;
- verification profile;
- engine safety minima.

10.20.2 Calibration outputs

- minimum retrieval quality;
- minimum target-negative margin;
- minimum local matches and inliers;
- minimum inlier ratio;
- maximum reprojection error;
- minimum spatial coverage;
- accepted pose uncertainty;
- minimum localized frames;
- temporal agreement threshold;
- target visibility threshold;
- checkpoint-specific parameters;
- predicted difficult conditions;
- reliability grade.

10.20.3 Profile behavior

Balanced prioritizes broad legitimate viewpoint support while retaining all safety gates.

Strict narrows pose and evidence tolerances.

Story-Critical maximizes false-positive protection and demands stronger negative and temporal evidence.

Custom may tune details but may not disable mandatory safety minima without Engineering-only warnings and an explicit non-automatic publication state.

10.21 B-4 work package 17: Reliability grades

10.21.1 Excellent

- no accepted negatives in test evidence;
- strong separation from confusers;
- high valid first-scan success;
- stable pose and temporal quality;
- suitable for automatic operation after shadow validation.

10.21.2 Good

- no accepted negatives;
- some valid viewpoints need guided retry;
- suitable for automatic use with guidance after shadow validation.

10.21.3 Needs Improvement

- insufficient positive coverage, weak negative margin, or difficult environmental failures;
- additional recording or region refinement required.

10.21.4 Unsafe

- any wrong location accepted;
- unstable or contradictory geometry;
- package integrity or calibration failure;
- automatic publication prohibited.

A grade is evidence, not marketing decoration. The report must show why it was assigned.

10.22 B-4 work package 18: Runtime package format

A published runtime package SHOULD include:

```
manifest.json
reference-global-index.bin
reference-local-features.bin
sparse-map.bin
stable-region-data.bin
hard-negative-index.bin
negative-map-data.bin (optional)
pose-region.json
verification-rules.json
guidance-map.json
calibration-report.json
compatibility.json
checksums.json
signature.json (when enabled)
thumbnail.webp
```

The package MUST exclude raw creator recordings unless a separate archival export is requested.

10.22.1 Package properties

- content-addressed files;
- full checksum manifest;
- schema version;
- immutable package version;
- model compatibility;
- hardware/runtime requirements;
- compressed size and uncompressed size;
- safe extraction rules;
- no executable scripts from community content;
- forward-compatible optional sections where practical.

10.23 B-4 work package 19: Local runtime execution

10.23.1 Runtime flow

Scan evidence from B-2

- > package compatibility check
- > quality gate
- > global target and negative retrieval

- > local matching
- > geometry and pose localization
- > spatial coverage
- > temporal consistency
- > checkpoint-specific rule
- > ambiguity veto
- > result and evidence report

10.23.2 Hardware paths

The runtime SHOULD support:

- NVIDIA GPU execution where available;
- compatible Windows GPU execution through an approved provider;
- CPU fallback;
- future cloud-assisted verification only as an explicit policy choice.

The engine must profile and constrain resource use so the game remains playable. It need not process every captured frame at full resolution.

10.24 B-4 work package 20: Runtime performance budgets

Initial engineering targets SHOULD include:

- bounded analysis resolution;
- selected rather than all captured frames;
- reference features precomputed at build time;
- package preloading before waypoint activation;
- no sustained high-rate inference when no scan is active;
- cancellation support;
- resource telemetry;
- graceful fallback if GPU allocation fails.

Exact time targets SHALL be measured on the hardware matrix. The UI should use a deliberate story-compatible analysis animation, but must not hide pathological runtime delay.

10.25 B-4 work package 21: Pilot waypoint program

At least three pilot waypoints are required:

10.25.1 Pilot A: Distinct exact landmark

A visually rich, fixed location expected to build reliably. Used to validate the complete pipeline.

10.25.2 Pilot B: Moderate natural scene

A location with rock, vegetation, shoreline, and several possible viewpoints. Used to validate pose tolerance and environmental variation.

10.25.3 Pilot C: Difficult confusable scene

A location with nearby lookalikes, reused props, or generic terrain. Used to validate negative separation and ambiguity rejection.

Pilot data **MUST** include new sessions not used to build the package.

10.26 B-4 work package 22: Test-dataset discipline

Each pilot **SHALL** maintain:

- reference/build dataset;
- calibration dataset;
- locked positive test dataset;
- locked hard-negative test dataset;
- boundary dataset;
- environmental dataset.

The locked dataset may not be silently moved into calibration after poor performance. Doing so requires a new version and a new locked evaluation set.

10.27 B-4 work package 23: Statistical reliability evidence

The report **SHOULD** include:

- number of positive attempts;
- first-scan pass rate;
- pass rate within two guided attempts;
- false-reject categories;
- number of negative attempts;
- false accepts;
- strongest rejected negative;
- confidence or margin distributions;
- hardware and condition breakdown.

For zero observed false accepts in N independent negative attempts, the report **MAY** include the approximate rule-of-three upper bound $3/N$ as context. It must not claim absolute safety from a small test set.

10.28 B-4 work package 24: Shadow-mode output

B-4 results are produced as recommendations:

```
{
  "outcome": "VERIFIED",
  "automaticProgressionAllowed": false,
  "failedGates": [],
  "evidenceSummary": {
    "localizedFrames": 7,
    "targetNegativeMargin": 0.14,
    "poseAccepted": true,
    "temporalAgreement": 0.92
  }
}
```

The story engine records but does not act automatically until B-5 wires the authorized mode.

10.29 B-4 testing strategy

10.29.1 Automated component tests

- deterministic frame preparation;
- global index build and query;
- negative margin logic;
- matcher adapters;
- geometric-estimation edge cases;
- pose-region membership;
- temporal trajectory validation;
- checkpoint rule modules;
- calibration reproducibility;
- package checksum validation;
- unsupported-version rejection;
- cancellation and worker restart.

10.29.2 Integration tests

- Studio build-input to runtime package;
- runtime package loaded by Companion;
- positive scan to shadow result;
- negative scan to rejection;
- invalid pose to rejection;
- package/model incompatibility;
- CPU fallback;
- missing artifact recovery;
- rebuild creates new immutable version.

10.29.3 Adversarial manual tests

- similar nearby rocks;
- correct target barely visible;
- wrong side of correct target;
- repeated architecture elsewhere;
- night, fog, rain, bright glare;
- teammate or held object blocking view;
- rapid camera movement;
- unusual FOV and resolution;
- very close and very far views;
- camera scan with insufficient context;
- attempt to reuse a recorded screenshot or static frame if relevant to threat model.

10.30 B-4 required demonstration

The demonstration SHALL show:

1. submit a real B-3 build-input package;
2. observe truthful build stages;
3. inspect generated reconstruction and calibration summary;
4. load the runtime package locally;

5. run an unseen valid scan from a position not identical to a reference frame;
6. obtain a correct pose and shadow VERIFIED result;
7. run a nearby similar wrong location and receive rejection;
8. run an accepted-area boundary case and receive the expected result;
9. show the failed gate for an ambiguous case;
10. repeat with a second and third pilot waypoint;
11. demonstrate CPU or alternate execution fallback;
12. prove raw creator recordings are absent from the runtime package;
13. reproduce the same calibration result from the same inputs and engine versions within documented tolerance.

10.31 B-4 exit gate

Phase B-4 Exit Gate

Phase B-4 is complete when three pilot waypoints can be built reproducibly, loaded locally, and evaluated through the full mandatory-gate pipeline in shadow mode. Unseen valid viewpoints must demonstrate practical tolerance, dangerous negatives must remain rejected, calibration and package contents must be auditable, and no real story may advance automatically yet.

10.31.1 Mandatory acceptance checklist

- ☐ Build pipeline consumes the B-3 package without manual folder surgery.
- ☐ Global target and hard-negative retrieval are implemented.
- ☐ Local matching and robust geometry are implemented.
- ☐ Sparse reconstruction and query camera pose are implemented where required.
- ☐ Accepted pose region and facing rules are enforced.
- ☐ Spatial evidence and temporal consistency are mandatory gates.
- ☐ Checkpoint-specific rule modules exist.
- ☐ Ambiguity veto is enforced.
- ☐ Calibration is per waypoint and reproducible.
- ☐ Runtime packages are immutable, checksummed, and exclude raw recordings.
- ☐ Three pilot waypoints pass locked positive and negative evaluation.
- ☐ No weighted score can override a failed mandatory gate.
- ☐ Automatic story progression remains disabled.

10.32 B-4 principal risks and mitigations

Risk	Consequence	Required mitigation
One model appears strong on pilot A and becomes hard-coded	Brittle architecture	Adapter interfaces and three diverse pilots
Positive accuracy tuned by weakening negative protection	Story-destroying false accepts	Mandatory negative margin, geometry, pose, ambiguity veto, Unsafe grade
Reconstruction fails on generic scenes	Excessive false negatives	Better capture guidance, submaps, planar/context hybrid, checkpoint-specific strategies

Risk	Consequence	Required mitigation
GPU inference harms game performance	Unusable player experience	Selected frames, downscaling, precomputation, profiling, fallback
Calibration overfits test data	Inflated reliability claim	Separate build, calibration, and locked test datasets
Research dependency changes break packages	Compatibility failure	Engine/model versions, package manifest, adapter isolation, reproducible builds
Raw creator footage leaks into release package	Privacy and size problem	Package-content validation and explicit archival separation

10.33 B-4 Codex completion evidence

Codex MUST provide:

- engine module diagram;
- build-stage implementation report;
- package schema and sample contents;
- pilot dataset inventory;
- calibration and locked-test reports;
- false-positive and false-negative analysis;
- runtime performance by hardware path;
- reconstruction and pose visualizations;
- model and dependency version lock;
- shadow-mode demonstration recording;
- known difficult conditions and exact creator guidance;
- rollback and package invalidation procedure.

Phase B-5: Player, Story, and Captain Integration

11.1 Phase mission

Integrate real Vision Waypoint verification into the Tall Tale experience without compromising story pacing, state integrity, recovery, privacy, or Captain control. This phase turns the B-4 shadow engine into a governed story capability and connects successful verification to the already completed Phase A presentation system.

At completion, a real player can reach an active waypoint, hold Inspect Surroundings, receive truthful scan feedback, obtain a verified result, trigger the configured Phase A reveal exactly once, and continue the story. Incorrect, ambiguous, and low-quality attempts remain safe and understandable. The Captain can inspect, confirm, override, or recover when reality behaves like reality.

11.2 In scope

- Player hold-to-scan UX;
- waypoint activation and package readiness;
- story-event contracts;
- exact-once progression;
- Phase A transition/cinematic triggering;
- guided retry mapping;
- Captain live status and diagnostics;
- shadow, Captain-confirmed, and automatic modes;
- manual override and audit trail;
- offline package caching and story continuity;
- cross-device/session synchronization;
- attempt history and improvement capture;
- privacy controls in the Player and Captain surfaces;
- user-facing failure and recovery states;
- field validation of pilot and first production waypoints.

11.3 Out of scope

- continuous passive recognition;
- public community package marketplace;
- full live AR compositing;
- removal of Captain fallback;
- automatic retraining from every attempt;
- support for unverified arbitrary game versions.

11.4 B-5 work package 1: Story-block integration

11.4.1 Vision Verification block

The Studio story editor SHALL include a block that references a published Vision Waypoint version.

Recommended block information:

Vision Waypoint

```
Waypoint: Sailor's Bounty Message Rock
Version: 3
Verification profile: Story-Critical
Operation mode: Shadow / Captain Confirmed / Automatic
Player interaction: Hold Inspect Surroundings
On success: emit waypoint.verified
Captain fallback: Enabled
```

11.4.2 Event separation

The Vision Waypoint verifies a condition. The story graph determines the response.

Example:

```
[Vision Waypoint Verified]
  |
  v
[Complete Objective]
  |
  v
[Play Phase A Scene Transition]
  |
  v
[Reveal Carved Message]
  |
  v
[Play Captain Audio]
  |
  v
[Unlock Journal Entry]
  |
  v
[Activate Next Objective]
```

The verifier MUST NOT embed story-specific cinematic files in its decision logic.

11.4.3 Required events

```
vision.waypoint_activated
vision.package_required
vision.package_ready
vision.scan_started
vision.scan_progress
vision.scan_canceled
vision.scan_completed
```

```

vision.verification_succeeded
vision.verification_insufficient
vision.verification_rejected
vision.verification_ambiguous
vision.verification_error
vision.captain_confirmed
vision.captain_rejected
vision.captain_override
vision.story_effect_started
vision.story_effect_completed

```

Events that can change story state **MUST** carry an idempotency key and expected story-stage version.

11.5 B-5 work package 2: Waypoint activation and package preparation

When a story stage containing a Vision Waypoint becomes active:

1. resolve the exact published waypoint version;
2. verify package compatibility with the local engine;
3. download or locate the package;
4. validate checksums and signature where enabled;
5. preload required runtime indexes within resource limits;
6. verify Companion capability;
7. report package-ready state to Player and Captain;
8. do not display a usable scan control until the system can genuinely process it.

11.5.1 Failure handling

If the package is unavailable or incompatible:

- Player sees a story-safe unavailable message;
- Captain sees the technical reason and options;
- automatic progression is disabled;
- manual Captain fallback remains available if permitted;
- the story state does not advance;
- a compatible package or engine update may be requested.

The interface **MUST NOT** show “Ready” while the package is still downloading or incompatible. Interface truth remains annoyingly important.

11.6 B-5 work package 3: Player prompt and interaction design

11.6.1 Activation presentation

The story may reveal the scan interaction through narrative language such as:

“The compass grows restless. Hold Inspect Surroundings and slowly search the scene.”

The prompt **SHOULD** include the configured key or control and remain accessible through an objective panel.

11.6.2 Scan presentation

While scanning, the Player UI should show:

- real capture-active indicator;
- progress based on actual scan state;
- subtle guidance to move slowly;
- cancel instruction;
- source-loss or quality warning if known in time;
- a deliberate cinematic treatment consistent with the journal/compass experience;
- reduced-motion alternative.

A long, motionless ten-second opacity fade remains prohibited because even pirates deserve better.

11.6.3 Completion transition

On release or automatic scan completion:

- input prompt changes to analysis state;
- animation may continue while B-4 evaluates;
- result state is derived from actual engine outcome;
- duplicate input is disabled until result or safe timeout;
- cancellation remains possible where processing supports it.

11.7 B-5 work package 4: Result-to-guidance mapping

The internal engine reason maps to story-safe player guidance.

11.7.1 Verified

Possible text:

“The compass locks into place.”

The configured success flow begins immediately or after a deliberate confirmation beat.

11.7.2 Insufficient visual evidence

Messages must be actionable without revealing the answer:

Technical cause	Player guidance example
Excessive blur	“Move more slowly while the compass is listening.”
Too close / target fills frame	“Step back and inspect more of the landmark.”
Target too small	“Move closer and keep the landmark in view.”
Facing near target but off-screen	“The compass stirs. Slowly inspect the area around you.”
Heavy obstruction	“Clear your view and inspect again.”
Too dark / low detail	“Find a clearer view or bring more light to the scene.”
Too little camera movement	“Slowly look across the surroundings while holding the compass.”
Too much camera movement	“Turn more gently so the compass can settle.”

11.7.3 Ambiguous

“The compass stirs, but cannot settle.”

The UI may suggest including more surrounding landmarks.

11.7.4 Not at target

“The compass remains silent.”

Do not reveal the nearest matching wrong location.

11.7.5 System error

The Player receives a calm recovery message while the Captain sees the technical details.

11.8 B-5 work package 5: Exact-once story progression

11.8.1 Requirements

- a verification attempt has a unique ID;
- a success result can be retried without duplicating side effects;
- story progression checks the expected active stage;
- a stale result from a prior stage is rejected;
- Captain override uses a separate auditable event;
- Phase A cinematic completion is tracked independently from verification success;
- reconnection does not replay the reveal unless the story rules explicitly support replay;
- Player history records the completed waypoint and associated story version.

11.8.2 Suggested transaction sequence

Verification succeeded

- > persist attempt outcome
- > claim idempotency key
- > confirm active story stage/version
- > persist waypoint completion
- > emit story transition event
- > launch presentation
- > persist presentation completion
- > activate next stage

The system must recover if the app closes between any two steps.

11.9 B-5 work package 6: Phase A reveal integration

The existing passive presentation system becomes the governed success renderer.

Supported presentation types may include:

- crossfade from captured scene to edited image;
- pre-rendered WebM cinematic;
- layered 2.5D reveal;
- Three.js artifact scene;
- audio and journal sequence;
- modified view selected from the nearest reference angle;
- later external-AR surface effect through the reserved interface.

11.9.1 Captured-scene use

If the reveal uses the final scan frame:

- Player consent and privacy policy must cover transient use;
- the frame is passed directly to the presentation pipeline;
- it is not retained afterward by default;
- the transition handles different aspect ratios and resolutions;
- the scene must not expose diagnostic overlays.

11.9.2 View-matched asset selection

Where the waypoint includes several edited viewpoint assets, the engine may provide the closest verified reference orientation. The story renderer selects the corresponding asset without changing the verification result.

11.10 B-5 work package 7: Captain live dashboard

11.10.1 Live status

Captain sees:

- active player and story stage;
- active waypoint and version;
- package readiness;
- current operation mode;
- scan in progress;
- capture-health summary;
- analysis state;
- final recommendation;
- attempt count and recent guidance.

11.10.2 Evidence summary

For each attempt:

- outcome;
- failed gates;
- usable/localized frame counts;
- target-negative margin;
- pose accepted or rejected;
- target visibility;
- temporal agreement;
- closest target reference;
- closest hard negative;
- runtime duration;
- model/package compatibility;
- retained-frame availability if explicit diagnostics were enabled.

11.10.3 Captain actions

- Confirm engine recommendation;
- Reject engine recommendation;
- Approve current stage manually;

- Trigger configured reveal manually;
- Ask player to rescan;
- pause vision;
- change operation mode where authorized;
- save attempt as potential positive;
- save attempt as hard negative;
- create diagnostic note;
- report package as outdated or unreliable.

Actions **MUST** require appropriate Captain permission and create audit history.

11.11 B-5 work package 8: Operation modes

11.11.1 Development

- available only to authorized developers/creators;
- detailed diagnostics visible;
- no automatic story progression;
- package may be incomplete.

11.11.2 Shadow

- engine processes real scans;
- Player behavior may remain manual or Captain-driven;
- recommendation is stored;
- Captain records whether recommendation was correct;
- automatic progression disabled.

11.11.3 Captain Confirmed

- engine recommendation appears to Captain;
- Captain confirms success before progression;
- useful when reliability is promising but not sufficiently validated.

11.11.4 Automatic

- verified outcome advances the story automatically;
- allowed only for a compatible published version meeting reliability and shadow criteria;
- Captain fallback remains available;
- mode can be revoked if field evidence reveals a safety problem.

11.11.5 Promotion requirements

Promotion **SHOULD** consider:

- reliability grade;
- number of shadow attempts;
- confirmed positive success;
- zero confirmed false accepts;
- representative hardware and condition coverage;
- package and game compatibility;
- creator or owner approval.

11.12 B-5 work package 9: Manual fallback and recovery

The Captain fallback is a governed fault-tolerance mechanism.

11.12.1 Manual approval reasons

- visual engine failed under unusual weather;
- capture unavailable;
- package incompatible during a private event;
- correct player state independently confirmed;
- accessibility accommodation;
- time-sensitive story recovery.

11.12.2 Requirements

- reason must be selected or entered;
- override identifies Captain and timestamp;
- override does not relabel the vision attempt as verified;
- story history records manual completion;
- override may be added to improvement review;
- public analytics distinguish automatic and manual completion.

11.13 B-5 work package 10: Offline package caching

11.13.1 Pre-adventure preparation

Before play, the desktop app SHOULD provide a readiness check:

- story package downloaded;
- waypoint packages downloaded;
- media assets downloaded;
- checksums valid;
- model runtime available;
- Companion healthy;
- required storage available;
- offline mode tested.

11.13.2 Offline story behavior

- scans and verification run locally;
- story progression persists locally;
- Phase A media plays from cache;
- Captain on the same device or local session remains usable according to product design;
- pending events sync when connectivity returns;
- identity and authorization rules remain safe.

11.13.3 Synchronization

Use idempotent event records with:

- event ID;
- session ID;

- story version;
- previous and resulting stage version;
- device ID;
- timestamp;
- verification or override reference;
- sync state.

Conflicts **MUST** be surfaced rather than silently choosing whichever device shouted last.

11.14 B-5 work package 11: Web-paired runtime

For users who run the Player in a browser or PWA:

- browser connects to the paired Companion;
- required waypoint package may reside in the Companion cache;
- scan progress flows through the secure local session;
- result flows to the shared story engine;
- page refresh can restore the active attempt state;
- loss of pairing pauses the interaction without advancing.

The browser experience should match desktop behavior, except native controls and offline depth may differ.

11.15 B-5 work package 12: Attempt history and field learning

11.15.1 Attempt record

Store metadata for:

- waypoint/version;
- story/session/player context;
- operation mode;
- outcome;
- failed gates;
- hardware path;
- duration;
- Captain judgment if any;
- whether diagnostic frames were retained with consent;
- whether the attempt was promoted into a future draft dataset.

11.15.2 Improvement actions

The authorized creator may select:

- Add as Valid Positive;
- Add as Boundary Example;
- Add as Hard Negative;
- Ignore;
- Mark Capture Problem;
- Mark Package Outdated.

This creates or updates a new draft version. It never mutates the published package used during the attempt.

11.16 B-5 work package 13: Privacy controls during play

Player settings **MUST** include:

- current selected window;
- Vision Active status;
- pause/resume;
- transient-processing explanation;
- diagnostic retention opt-in;
- delete retained diagnostic attempts;
- local/cloud processing status;
- pairing and device control.

Captain must not be able to silently enable raw-frame retention against the Player's policy without explicit consent.

11.17 B-5 work package 14: Player and Captain notifications

Notifications should be event-driven and restrained:

- package downloading;
- package ready;
- Companion disconnected;
- capture paused;
- scan ready;
- scan quality issue;
- verification result;
- Captain override applied;
- offline state and pending sync.

No notification should claim success before the story transaction commits.

11.18 B-5 work package 15: First production waypoint rollout

After pilot validation:

1. select one story-critical but visually distinctive waypoint;
2. author a production version through Studio;
3. complete locked testing;
4. deploy in Shadow mode;
5. run Captain-confirmed sessions;
6. review all attempts;
7. promote to Automatic only after criteria pass;
8. retain a rollback to Captain-confirmed mode.

The first production rollout should not use the most generic beach in the game, because the project is ambitious enough without attempting to prove sand has an identity.

11.19 B-5 testing strategy

11.19.1 Automated

- story-event idempotency;
- stale result rejection;
- package-ready gating;
- Player guidance mapping;
- Captain permissions;
- override audit history;
- phase A trigger exactly once;
- crash between verification and reveal;
- offline event persistence;
- sync conflict handling;
- browser reconnect;
- package incompatibility;
- operation-mode promotion rules;
- immutable published package use.

11.19.2 End-to-end scenarios

1. Correct first scan succeeds and reveal plays.
2. Blurry scan gives guidance; second scan succeeds.
3. Similar wrong location is rejected without story leak.
4. Ambiguous scan asks for wider context.
5. Companion disconnects during scan and story remains unchanged.
6. App closes after verification but before reveal; recovery resumes safely.
7. Same result event is delivered twice; story advances once.
8. Captain manually approves after capture failure.
9. Offline desktop completes stage and syncs later.
10. Browser refreshes during active waypoint and restores state.
11. Outdated package disables automatic progression.
12. Diagnostic retention remains off unless Player consents.

11.20 B-5 required demonstration

The demonstration SHALL show:

1. author and publish a production candidate waypoint version;
2. attach it to a real story graph;
3. download and validate its package;
4. activate the waypoint in Player mode;
5. perform a poor scan and receive appropriate guidance;
6. perform a correct scan and obtain a verified result;
7. trigger the configured Phase A cinematic and journal/story progression exactly once;
8. perform a similar wrong-location scan and confirm no progression;
9. inspect the attempts in Captain dashboard;
10. complete one attempt through Captain confirmation and one through manual override;
11. demonstrate Shadow and Automatic mode controls;
12. demonstrate offline completion and later synchronization;
13. show that the published waypoint version remains unchanged after adding a field attempt to a new draft.

11.21 B-5 exit gate

Phase B-5 Exit Gate

Phase B-5 is complete when real verification is fully integrated with Player interaction, Captain oversight, the story state machine, Phase A presentation, package readiness, offline continuity, and exact-once progression. At least one production candidate waypoint must complete the Shadow-to-Captain-confirmed path, and automatic operation must be demonstrably safe and revocable.

11.21.1 Mandatory acceptance checklist

- ☐ Vision Verification story block references immutable published versions.
- ☐ Package readiness is truthful and enforced.
- ☐ Hold-to-scan uses real capture and B-4 results.
- ☐ Player guidance maps to technical failure reason without revealing the solution.
- ☐ Correct verification triggers Phase A presentation exactly once.
- ☐ Wrong and ambiguous attempts do not advance the story.
- ☐ Captain dashboard shows evidence and controls.
- ☐ Overrides are permissioned and audited.
- ☐ Shadow, Captain Confirmed, and Automatic modes are enforced.
- ☐ Offline desktop operation and synchronization pass.
- ☐ Browser pairing can reconnect safely.
- ☐ Field attempts can improve a new draft without mutating published history.
- ☐ Privacy controls remain visible and respected.

11.22 B-5 principal risks and mitigations

Risk	Consequence	Required mitigation
Duplicate success event advances story twice	Broken progression and repeated reveal	Idempotency key, expected stage version, transaction log
Cinematic starts before persistence	Crash loses completion state	Commit verification/story transition before or with recoverable presentation event
Guidance reveals exact target	Puzzle spoiled	Story-safe mapping reviewed per waypoint type
Captain override hides engine failure	Reliability never improves	Separate override classification, required reason, improvement workflow
Offline and online state diverge	Conflicting player history	Event-based sync, version checks, explicit conflict handling
Automatic mode enabled too early	False positive ruins story	Promotion policy, shadow evidence, revocable mode
Player does not understand capture state	Privacy concern	persistent status, selected-window display, pause and consent controls

11.23 B-5 Codex completion evidence

Codex MUST provide:

- story-event sequence diagrams;
- Player and Captain screenshots/recording;
- exact-once and crash-recovery test output;
- offline synchronization report;
- operation-mode promotion implementation;
- guidance mapping catalog;
- Phase A integration evidence;
- privacy-control demonstration;
- field-attempt draft improvement demonstration;
- known limitations and rollback from Automatic to Captain Confirmed.

Phase B-6: Hardening and Public Creator Release

12.1 Phase mission

Turn the complete working system into a stable product that another person can install, understand, author with, test, publish from, and recover without direct developer assistance. This phase validates reliability across hardware and game conditions, secures the native and package surfaces, finalizes installation and updates, documents the product, and establishes long-term support and compatibility rules.

At completion, Phase B will be eligible for Stable release to invited external creators and players. Public community sharing may remain limited, but the product itself must no longer depend on the project owner remembering which undocumented ritual fixes a failed waypoint build.

12.2 In scope

- full regression and reliability program;
- adversarial negative testing;
- hardware and display matrix;
- performance budgets;
- native installer;
- application and package updating;
- signing and release provenance;
- crash recovery and rollback;
- security review;
- privacy validation;
- creator usability testing;
- Player and Captain usability testing;
- documentation and onboarding;
- support diagnostics;
- compatibility and deprecation policy;
- release channels;
- release readiness dashboard;
- first external-creator cohort;
- phase completion report and operational handoff.

12.3 Out of scope

- unrestricted public marketplace;
- moderation and monetization systems;

- continuous automatic scanning;
- full live AR;
- support for every Windows hardware configuration ever manufactured by a civilization with electricity;
- guarantees that future game updates will never require waypoint retesting.

12.4 B-6 work package 1: Complete regression suite

The project SHALL establish automated and manual regression suites spanning all prior phases.

12.4.1 Domain and contract regression

- waypoint identity and versions;
- publication immutability;
- lifecycle transitions;
- story references;
- protocol compatibility;
- feature flags;
- package checksums;
- event idempotency;
- audit history;
- permissions.

12.4.2 Capture regression

- source selection;
- resize;
- close/reopen;
- minimize/restore;
- multi-monitor move;
- frame quality;
- ring-buffer bounds;
- no-disk Player default;
- pairing expiration;
- desktop integrated connection;
- hotkey configuration;
- capture interruption recovery.

12.4.3 Authoring regression

- new waypoint wizard;
- resumability;
- recording categories;
- stable-region editing;
- accepted-region editing;
- data-health validation;
- build-input schema;
- version comparison;
- archived usage;
- accessibility.

12.4.4 Vision regression

- build reproducibility;
- retrieval and negative separation;
- geometry;
- pose localization;
- spatial coverage;
- temporal consistency;
- checkpoint rules;
- calibration;
- package loading;
- CPU/GPU paths;
- locked test sets.

12.4.5 Story regression

- package activation;
- Player scan;
- guidance;
- exact-once reveal;
- Captain actions;
- operation modes;
- offline completion;
- sync;
- field improvement;
- privacy settings.

12.5 B-6 work package 2: Reliability and adversarial test program

12.5.1 Negative sweep

For each Story-Critical waypoint:

1. activate the waypoint in test mode;
2. visit nearby similar locations;
3. test invalid sides and boundaries;
4. test repeated props elsewhere;
5. test visually generic conditions;
6. retain near-pass metadata;
7. mine the strongest negatives into a new draft where appropriate;
8. rebuild and rerun the locked test suite;
9. document remaining risk.

12.5.2 Positive robustness

Test valid players under:

- alternate approaches;
- close, medium, and far accepted positions;
- different camera pitches;
- supported FOV range;
- 1080p and 1440p;

- different graphics quality;
- daylight, sunset, night, fog, storm, glare, and lantern light where applicable;
- partial obstruction;
- different movement speed;
- different supported hardware path.

12.5.3 Release reliability targets

Each published automatic waypoint SHOULD meet at least:

- zero confirmed false accepts in its locked negative release dataset;
- high first-scan positive success from ordinary accepted viewpoints;
- very high success within two guided attempts;
- no unexplained or nondeterministic result changes under repeated identical evidence;
- documented unsupported or difficult conditions;
- successful shadow and Captain-confirmed field use.

The exact numeric release minimums SHALL be governed by verification profile and evidence volume. Any claim must include dataset size and conditions.

12.6 B-6 work package 3: Hardware and display compatibility matrix

12.6.1 Required classes

- supported NVIDIA GPU path;
- supported non-NVIDIA Windows GPU path where available;
- CPU fallback;
- integrated and discrete graphics combinations;
- 16 GB and 32 GB system memory where practical;
- 1080p and 1440p;
- common 16:9 and 16:10 aspect ratios;
- ultrawide as supported or explicitly limited;
- single and multiple monitors;
- borderless and windowed game modes;
- common Windows scaling settings.

12.6.2 For each configuration record

- capture success;
- selected frame throughput;
- build availability if local;
- runtime verification duration;
- peak CPU, GPU, and memory usage;
- game performance effect;
- package-load time;
- installer/updater behavior;
- known limitations.

12.6.3 Capability tiers

The product MAY classify devices:

- Full Local Build and Runtime;
- Local Runtime, Cloud-Assisted Build Recommended;
- CPU Runtime with Reduced Performance;
- Unsupported for Vision, Web Features Available.

The classification must explain the limitation rather than implying personal failure on behalf of the computer.

12.7 B-6 work package 4: Performance hardening

12.7.1 Budgets

Define and measure budgets for:

- Companion idle CPU/GPU usage;
- active capture usage;
- scan memory usage;
- verification latency;
- package preload time;
- Studio recording upload or processing;
- local build duration by hardware tier;
- application startup;
- update size;
- offline package size.

12.7.2 Required optimizations

- no continuous heavy inference when scan inactive;
- precomputed reference features;
- analysis downscaling;
- bounded selected frame count;
- lazy package loading;
- cache eviction rules;
- worker cancellation;
- GPU failure fallback;
- no unbounded diagnostic storage;
- no rendering animation that blocks verification worker progress.

12.8 B-6 work package 5: Installer and application packaging

The Windows desktop release MUST include:

- clear product name and icon;
- per-user installation where practical;
- explicit installation location and data directories;
- required runtime dependencies bundled or installed safely;
- uninstaller;
- preservation or removal choices for user data;
- version and build ID;
- release channel;
- logs and diagnostic path;
- compatibility with supported Windows versions;

- no requirement for administrator rights unless a documented platform need exists.

12.8.1 Installer tests

- clean machine install;
- upgrade from prior release;
- failed upgrade rollback;
- uninstall while preserving data;
- complete uninstall;
- interrupted installation;
- low disk space;
- antivirus/SmartScreen behavior;
- multiple Windows users;
- corrupted installer detection.

12.9 B-6 work package 6: Application signing and provenance

Public or external distribution **SHOULD** use signed application binaries and signed update metadata.

The release process **MUST** record:

- source commit;
- build ID;
- dependency lock state;
- build environment;
- generated artifact checksums;
- signing identity;
- release channel;
- publication timestamp;
- rollback target.

Secrets used for signing **MUST** remain outside the repository and Codex logs.

12.10 B-6 work package 7: Updater and rollback

12.10.1 Update channels

Development

Creator Preview

Stable

Potential future channel:

Experimental Vision Models

12.10.2 Update components

- desktop application;
- Companion service;
- vision worker;
- model bundle;
- package schema compatibility metadata;
- optional content-package updates.

12.10.3 Updater requirements

- verify signed update metadata;
- verify checksums;
- download atomically;
- install without corrupting current working release;
- retain rollback capability;
- communicate restart requirements;
- detect incompatible waypoint packages before activation;
- avoid changing a model silently during an active Tall Tale;
- support deferred updates;
- preserve user data and cached published packages.

12.10.4 Model update policy

A new model bundle **MUST NOT** retroactively alter an immutable published waypoint's declared behavior without compatibility testing. Options include:

- package pins compatible model family/version;
- new runtime validates equivalence before use;
- waypoint is marked Needs Retest;
- creator publishes a new waypoint version.

12.11 B-6 work package 8: Runtime package security

Vision Waypoint packages **MUST** be treated as untrusted content when imported or shared.

12.11.1 Requirements

- schema validation;
- checksum verification;
- signature verification for trusted scopes;
- path traversal prevention;
- decompression size limits;
- no executable scripts;
- no arbitrary native libraries;
- no code evaluation from manifests;
- bounded model or tensor sizes;
- safe image parsing;
- quarantine on failure;
- clear source and trust scope.

A community package should contain data and governed configuration, not a tiny pirate-themed remote-code-execution festival.

12.12 B-6 work package 9: Localhost and native security review

Review:

- pairing origin restrictions;
- session-secret storage;
- request authentication;

- anti-replay behavior;
- rate limits;
- command allowlist;
- frontend/native privilege boundary;
- log redaction;
- diagnostic retention;
- update channel;
- package extraction;
- cloud-assisted upload consent;
- cross-account device pairing;
- logout and revocation.

Perform threat-model scenarios:

- malicious webpage on the same machine;
- stolen pairing code;
- stale paired browser session;
- modified runtime package;
- poisoned update metadata;
- compromised creator package;
- excessive diagnostic retention;
- local user attempting to read another user's data.

12.13 B-6 work package 10: Privacy validation

12.13.1 Required verification

- Player scans do not persist by default;
- diagnostic frame retention is explicit and visible;
- creator recordings have deletion controls;
- cloud-assisted processing has separate consent;
- no raw frames appear in routine logs or analytics;
- Captain permissions do not bypass Player privacy policy;
- pairing revocation stops access;
- account deletion and local-data removal are documented;
- retention durations are implemented, not merely written in a policy.

12.13.2 Privacy documentation

Provide plain-language explanations for:

- what is captured;
- why it is captured;
- where processing occurs;
- what is retained;
- what may be uploaded;
- how to pause and delete;
- what the Captain can see;
- how the game itself remains untouched.

12.14 B-6 work package 11: Game-safety and policy review

Before broader release:

- verify the current game and platform rules;
- document that the product uses read-only external window capture;
- confirm there is no injection, memory reading, game-file modification, automation, competitive overlay, or visibility alteration;
- document supported presentation on a separate app/window;
- disable any feature that would cross the established safety boundary;
- preserve a kill switch for vision operation if policy uncertainty arises.

The product architecture should remain conservative even if a more aggressive shortcut appears technically possible.

12.15 B-6 work package 12: Creator usability validation

12.15.1 External creator cohort

Recruit users who did not participate in implementation. Include at least:

- one technically comfortable creator;
- one ordinary game/story creator;
- one user with minimal machine-learning or development knowledge.

12.15.2 Unassisted task sequence

Each creator should attempt to:

1. install or open the product;
2. connect the Companion or use desktop integrated mode;
3. create a Vision Waypoint;
4. choose the correct type;
5. record target coverage;
6. define accepted area;
7. record hard negatives;
8. resolve a data-health warning;
9. build and test the waypoint;
10. attach it to a Tall Tale;
11. publish a version;
12. run a Player scan;
13. inspect Captain evidence;
14. improve the waypoint from an attempt.

12.15.3 Observation rule

Developers SHOULD not coach unless the participant is blocked or safety requires intervention. Every repeated verbal explanation is a candidate interface defect or missing documentation.

12.15.4 Usability measures

- task completion;
- time and confusion points;
- number of incorrect recordings;

- number of abandoned steps;
- comprehension of target versus hard negative;
- comprehension of privacy state;
- ability to recover from an error;
- confidence in reliability grade;
- ability to find documentation and diagnostics.

12.16 B-6 work package 13: Player and Captain usability validation

Player testing must confirm:

- scan prompt is understandable;
- interaction works during normal gameplay;
- guidance is useful without spoiling the clue;
- capture state feels trustworthy;
- successful transition is cinematic and not abrupt;
- retries are not humiliating or repetitive;
- privacy controls are discoverable.

Captain testing must confirm:

- active state is obvious;
- evidence summary is understandable;
- override is safe and intentional;
- failure reason leads to useful action;
- improvement workflow is discoverable;
- diagnostics do not overwhelm ordinary operation.

12.17 B-6 work package 14: Documentation set

12.17.1 Creator guide

- installation and connection;
- waypoint types;
- recording best practices;
- accepted-area capture;
- hard negatives;
- stable regions;
- build and reliability grades;
- testing;
- publication and versions;
- field improvement;
- common problems.

12.17.2 Player guide

- installation or browser pairing;
- selecting the game window;
- hold-to-scan;
- guidance messages;
- privacy and pause controls;

- offline behavior;
- troubleshooting.

12.17.3 Captain guide

- active waypoint monitoring;
- shadow and automatic modes;
- evidence interpretation;
- confirmation and override;
- recovery;
- attempt improvement;
- audit history.

12.17.4 Developer/operator guide

- architecture;
- processes and ports;
- package schemas;
- model bundles;
- logs and diagnostics;
- build pipeline;
- security boundaries;
- release process;
- rollback;
- compatibility and deprecation.

12.18 B-6 work package 15: Support and diagnostics

The product SHOULD provide:

- one-click diagnostic bundle creation;
- clear bundle contents before export;
- frame-free default bundle;
- optional consented visual evidence;
- build-job diagnostic report;
- package compatibility report;
- capture health test;
- hardware capability test;
- update health test;
- cache validation and repair;
- safe reset of Companion pairing;
- link from error messages to relevant local help.

12.19 B-6 work package 16: Compatibility and deprecation policy

12.19.1 Compatibility dimensions

- application version;
- protocol version;
- runtime package schema;
- model bundle;

- capture service;
- operating system;
- game version/environment change;
- waypoint package status.

12.19.2 Package statuses

COMPATIBLE

COMPATIBLE_WITH_WARNING

NEEDS_RETEST

OUTDATED

INCOMPATIBLE

REVOKED

12.19.3 Deprecation rules

- published history remains readable;
- active stories receive warnings before package removal;
- automatic mode may be revoked without deleting the waypoint;
- creators can migrate to a new version;
- the system explains why retesting is required;
- compatibility changes are documented in release notes.

12.20 B-6 work package 17: Release channels and readiness dashboard

The Studio/administration surface SHOULD show:

- current application release;
- Companion version;
- model bundle;
- package schema;
- pilot and production waypoint status;
- regression status;
- security-review status;
- installer status;
- documentation status;
- open release blockers;
- rollback target.

A release cannot be promoted to Stable while mandatory blockers remain.

12.21 B-6 work package 18: Operational monitoring

Collect privacy-preserving operational metrics such as:

- application and Companion crash rates;
- package-load failures;
- verification runtime duration;
- outcome category counts;
- retry counts;
- override counts;

- update success;
- hardware capability distribution.

Do not collect raw gameplay frames or sensitive story content as routine telemetry.

Metrics must distinguish:

- automatic verification;
- Captain confirmation;
- manual override;
- capture/system failure;
- player cancellation.

12.22 B-6 work package 19: Stable release procedure

1. freeze release candidate;
2. pin dependencies and model bundle;
3. run clean build;
4. execute automated regression;
5. execute hardware matrix smoke tests;
6. run security and privacy checklist;
7. validate installer and updater;
8. validate rollback;
9. validate documentation links;
10. validate first-run onboarding;
11. sign and checksum artifacts;
12. deploy to Creator Preview;
13. monitor external cohort;
14. resolve blockers in a new release candidate;
15. approve Stable;
16. publish release notes and known limitations.

12.23 B-6 required demonstration

The final Phase B demonstration SHALL show an external or uninvolved creator:

1. install the desktop application or use the supported web/Companion path;
2. complete first-run onboarding;
3. create and build a waypoint without terminal access;
4. resolve a reliability warning;
5. publish and attach the waypoint;
6. run a correct Player scan and Phase A reveal;
7. run an incorrect similar-location scan and remain rejected;
8. inspect Captain evidence;
9. improve a new draft from a field attempt;
10. perform an application update;
11. recover through rollback or repair test;
12. export a privacy-safe diagnostic bundle;
13. verify offline operation and later sync.

12.24 B-6 exit gate

Phase B-6 Exit Gate

Phase B is complete only when the entire product is reliable, secure, privacy-conscious, installable, updateable, documented, recoverable, and usable by people who did not build it. All prior phase gates must remain passing. Stable release requires explicit approval based on evidence, not merely the exhaustion of everyone involved.

12.24.1 Mandatory acceptance checklist

- ☐ Full automated regression passes.
- ☐ Hardware and display matrix is documented.
- ☐ Performance remains within approved budgets.
- ☐ Installer, updater, and rollback pass clean-machine testing.
- ☐ Release artifacts are versioned, checksummed, and signed where required.
- ☐ Native, localhost, package, and update security reviews pass.
- ☐ Player-frame privacy defaults are independently verified.
- ☐ External creators complete the authoring workflow without developer intervention.
- ☐ Player and Captain usability testing passes.
- ☐ Documentation exists for every role.
- ☐ Compatibility and deprecation policy is implemented.
- ☐ Support diagnostics are useful and privacy-safe.
- ☐ Stable release and rollback procedures are documented and rehearsed.
- ☐ The final six-phase completion report is committed.

12.25 B-6 principal risks and mitigations

Risk	Consequence	Required mitigation
Product works only on developer machines	Failed public use	Clean-machine installer tests and hardware cohort
Model or app update changes waypoint behavior	Silent regression	package/model compatibility, retest status, pinned versions, rollback
Community package contains unsafe content	Security incident	data-only schema, signature/checksum, safe extraction, no code execution
Usability testing is coached	False confidence	unassisted external tasks and observation notes
Privacy promises differ from implementation	Trust failure	automated no-retention tests and explicit audit
Support requires raw frames for every issue	Privacy and scale problem	rich metadata diagnostics, opt-in expiring visual evidence
Game update changes environment	degraded accuracy	compatibility status, field reports, Needs Retest workflow

12.26 B-6 Codex completion evidence

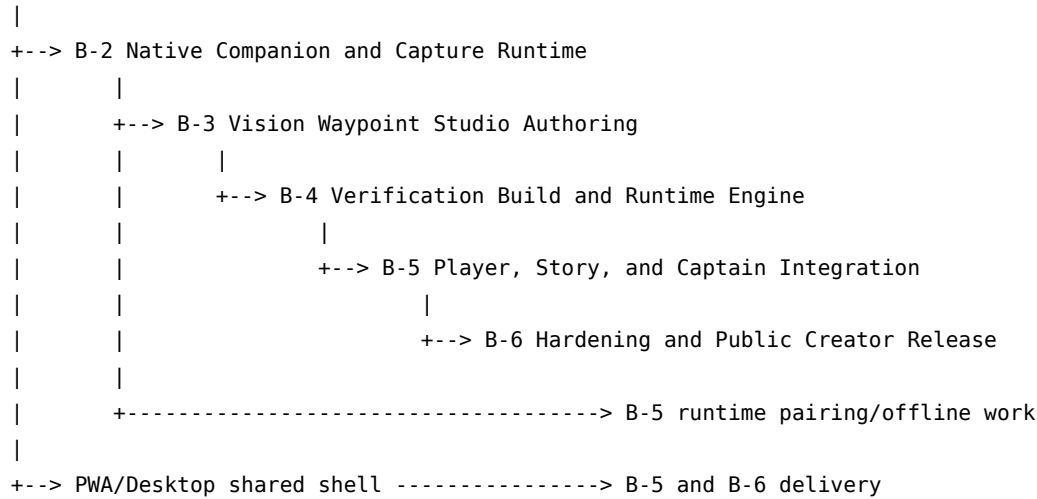
Codex MUST provide:

- master regression report;
- hardware/performance matrix;
- installer/updater/rollback evidence;
- artifact checksums and build provenance;
- security and privacy checklists;
- external usability test report;
- documentation inventory;
- compatibility/deprecation implementation;
- final release demonstration;
- Stable release notes;
- final Phase B completion report;
- open post-Phase-B backlog explicitly separated from incomplete Phase B requirements.

Cross-Phase Dependency and Critical Path

13.1 Dependency graph

B-1 Shared Platform Foundation



The phases are sequential at the acceptance-gate level, but preparation may overlap carefully:

- design exploration for B-3 may begin during B-2, but B-3 may not assume capture behavior that has not passed B-2;
- model benchmarking for B-4 may begin with research fixtures, but production package work must consume the real B-3 schema;
- installer exploration for B-6 may begin earlier, but Stable packaging cannot mask incomplete B-5 behavior.

13.2 Critical path items

The following items are critical and **SHOULD** receive early technical spikes before their phase begins:

1. safe Windows game-window capture and source-health detection;
2. shared frontend operation in the desktop shell;
3. browser-to-local Companion pairing security;
4. build-input package completeness;
5. reconstruction quality on natural Sea of Thieves scenes;
6. target-versus-hard-negative separation;
7. runtime performance while the game is running;
8. exact-once story progression and offline recovery;
9. signed update and model/package compatibility strategy.

A technical spike may inform the implementation, but it is not a substitute for the governed phase deliverable.

Phase Scope Control and Anti-Fragmentation Rules

14.1 Six phases means six phases

The project tracker may contain hundreds of issues. The public roadmap SHALL still contain six Phase B phases. Subdividing work is healthy; renaming every subdivision as a phase is not.

14.1.1 Valid decomposition

Phase B-4

Epic: Build orchestration
 Epic: Global retrieval
 Epic: Local matching
 Epic: Reconstruction
 Epic: Calibration
 Epic: Runtime package

14.1.2 Invalid decomposition

Phase B-4.1

Phase B-4.1A

Phase B-4.1A-Revision-2

Phase B-4.2 Pre-Phase

The latter is how project plans become geological strata.

14.2 Work-package completion

A work package can close when:

- code is integrated;
- tests pass;
- documentation is updated;
- its phase demonstration still works;
- no unresolved blocker is hidden as “future polish.”

A phase remains open until every mandatory exit item passes.

14.3 Deferral rules

A requirement may be deferred only when:

1. it is explicitly nonessential to the phase exit gate;
2. deferral does not create unsafe or fake behavior;
3. a later authorized phase clearly owns it;
4. the completion report names it;
5. the governing document does not require it earlier.

The following may not be deferred as polish:

- persistence;
- privacy controls;
- package integrity;
- hard-negative behavior;
- exact-once progression;
- error recovery;
- accessibility of critical controls;
- tests required by the exit gate.

Architecture Decision Record Policy

15.1 ADR triggers

An ADR is required for decisions such as:

- desktop framework selection or replacement;
- capture API selection;
- local process architecture;
- artifact-storage model;
- global retrieval family;
- primary and fallback matcher strategy;
- reconstruction engine;
- package-signing approach;
- update architecture;
- cloud-assisted build design;
- meaningful deviation from local-first processing;
- removal or weakening of a mandatory verification gate;
- changes to immutable-version rules.

15.2 ADR minimum template

Title

Status

Date

Context

Decision

Alternatives Considered

Consequences

Security and Privacy Effects

Creator and Player Effects

Compatibility and Migration

Rollback

Affected Governing Requirements

Approval

15.3 Model substitutions

A model or library may be replaced without rewriting the entire phase plan if:

- adapter contracts remain stable;
- package schema remains compatible or is versioned;
- all mandatory gates remain;
- calibration and locked tests rerun;
- performance and reliability are no worse;
- the substitution is recorded in an ADR and release notes.

Source Control, Build, and Release Discipline

16.1 Canonical source of truth

The canonical repository and branch **MUST** be identified before B-I. Codex must confirm it is operating in the correct worktree before modifying files. Multiple local copies **SHALL** not independently become “the real one” based on whichever has the freshest coffee stain.

16.2 Commit expectations

Commits **SHOULD** be:

- scoped to one logical change;
- descriptive;
- free of generated temporary data unless intentionally versioned;
- accompanied by migrations/tests where required;
- safe to revert;
- free of credentials and raw private recordings.

16.3 Build identification

Every runnable application or service **SHOULD** expose:

- semantic version;
- build ID;
- source commit;
- build timestamp;
- release channel;
- protocol version;
- package schema version;
- model bundle version where applicable.

16.4 Generated artifacts

Generated artifacts **MUST** be separated into:

- development intermediates;
- test fixtures;
- published runtime packages;

- release binaries;
- diagnostic bundles.

Their retention and source-control rules must be documented.

Testing Pyramid and Evidence Standard

17.1 Unit tests

Use for:

- domain rules;
- lifecycle transitions;
- geometry helpers;
- pose-region membership;
- frame-quality metrics;
- threshold logic;
- schema validation;
- event idempotency;
- permission checks.

17.2 Integration tests

Use for:

- database persistence;
- API contracts;
- Companion protocol;
- native capture session management;
- Studio package generation;
- build pipeline stages;
- runtime package loading;
- story transactions;
- offline synchronization.

17.3 End-to-end tests

Use for:

- browser paired to Companion;
- desktop integrated mode;
- creator recording to package;
- Player scan to result;
- result to Phase A reveal;

- Captain override;
- update and rollback;
- clean installation and first-run onboarding.

17.4 Visual and usability verification

Automated tests cannot prove that:

- guidance is understandable;
- progress is truthful to the user;
- the authoring wizard feels coherent;
- Captain evidence is readable;
- cinematic transitions preserve immersion;
- accessibility is adequate.

Each phase therefore requires visual inspection and manual demonstration.

17.5 Evidence hierarchy

A completion claim is strongest when supported by:

1. reproducible automated test;
2. deterministic artifact or report;
3. recorded end-to-end demonstration;
4. human usability observation;
5. documented manual steps.

A screenshot of a button is not evidence that the button works.

Reliability Governance

18.1 False-positive priority

A false positive can prematurely reveal story content. Therefore:

- ambiguity rejects;
- hard-negative comparison is mandatory;
- geometry and pose are mandatory where the waypoint requires them;
- automatic mode requires shadow evidence;
- a confirmed false accept revokes automatic eligibility until corrected.

18.2 False-negative mitigation

False negatives are managed through:

- broad valid pose coverage;
- multiple viewpoints;
- 3D localization instead of exact screenshot matching;
- environmental variants;
- guided retry messages;
- Captain confirmation and override;
- field-attempt improvement;
- per-waypoint calibration.

The engine must not weaken false-positive protection simply to make first-scan success look better.

18.3 No universal threshold

Thresholds are calibrated per waypoint and version. Global safety minima may exist, but a generic cave and a distinct carved door do not share identical evidence characteristics.

18.4 Confirmed false accept procedure

1. disable Automatic mode for affected version;
2. preserve the attempt evidence and audit record;
3. notify authorized creator/owner;
4. classify the confuser;
5. create a new draft version;

6. add hard-negative evidence;
7. rebuild and rerun locked tests;
8. complete Shadow and Captain-confirmed validation;
9. publish a new version;
10. migrate stories explicitly.

The published flawed version remains historical but may be revoked from new automatic use.

Security and Privacy Governance Across Phases

19.1 Threat boundaries

The system interacts with:

- game window pixels;
- local native process;
- browser or desktop frontend;
- user account and story backend;
- creator recordings;
- model and runtime packages;
- update infrastructure.

Every boundary needs authentication, validation, or both.

19.2 Minimum privacy requirements by phase

Phase	Privacy requirement
B-1	Data classes, retention fields, permissions, audit design
B-2	Selected-window consent, transient Player frames, visible capture state, secure pairing
B-3	Creator recording ownership, deletion, local/cloud choice, collaborative permissions
B-4	Runtime packages exclude raw recordings, logs exclude frames, diagnostic consent
B-5	Player controls, Captain limitations, field-attempt consent, offline protection
B-6	Independent validation, retention enforcement, documentation, export/delete support

19.3 No silent scope expansion

The Companion **MUST NOT** evolve from “capture the selected Sea of Thieves window” into “observe the entire desktop because it is easier.” Any broader capture mode requires explicit design, consent, and ADR, and should not be necessary for this product.

Data and Artifact Ownership

20.1 User-owned or project-owned data classes

- Tall Tales and story versions;
- Vision Waypoint stable identities;
- waypoint drafts and published versions;
- creator recordings;
- calibration and test reports;
- runtime packages;
- player progression;
- verification attempt metadata;
- optional retained diagnostic evidence;
- application logs;
- model bundles and engine artifacts.

20.2 Retention principles

- creator recordings persist until deleted, archived, or governed cleanup applies;
- published packages persist while referenced;
- runtime Player frames are transient by default;
- optional diagnostic frames expire according to policy;
- attempt metadata may persist for audit and reliability without raw frames;
- build intermediates may be cleaned after package validation while preserving reproducibility metadata;
- deleted accounts and shared assets require ownership-transfer or safe-removal rules.

Phase-by-Phase Deliverable Matrix

Deliverable	B-1	B-2	B-3	B-4	B-5	B-6
Shared web/desktop UI	Complete	Maintain	Maintain	Maintain	Production use	Harden
PWA foundation	Complete	Maintain	Maintain	Maintain	Offline shell expansion	Stable
Waypoint domain/versioning	Complete	Use	Use	Use	Use	Harden
Native capture	Mock	Complete	Use	Use	Production use	Harden
Authoring wizard	Skeleton	Connection hooks	Complete	Build/results integration	Story attachment	Usability hardening
Build engine	Contract only	Placeholder	Input package	Complete	Production packages	Harden
Verification runtime	Mock	Passthrough	Test UI	Complete shadow	Story integration	Stable
Captain diagnostics	Mock	Capture health	Authoring status	Engine evidence	Complete	Harden
Offline story use	Architecture	Capture local	Package planning	Local runtime	Complete	Stable
Installer/updater	Development shell	Dev distribution	Maintain	Maintain	Candidate	Complete

Master Phase Demonstration Sequence

The demonstrations form a chain. Each phase must preserve prior demonstrations.

22.1 B-1 demonstration

Mock waypoint -> persisted version -> mock verification -> mock reveal.

22.2 B-2 demonstration

Real window -> real scan evidence -> mock verification -> mock reveal.

22.3 B-3 demonstration

Studio authoring -> complete build-input package -> transitional build -> mock or partial result.

22.4 B-4 demonstration

Studio package -> real build -> real shadow verification on positives and negatives.

22.5 B-5 demonstration

Real verification -> exact-once Phase A reveal -> Captain/offline recovery.

22.6 B-6 demonstration

External creator installs, authors, publishes, plays, updates, and diagnoses the product.

Codex Phase Prompt Template

Every detailed Codex prompt SHOULD follow this structure.

23.1 1. Role and authority

State that Codex is implementing the named phase under TT-VISION-GOV-001 and TT-VISION-PHASE-B-001. Require inspection of the current repository before changes.

23.2 2. Current baseline

Include:

- canonical repository and branch;
- current version and build ID;
- prior phase completion evidence;
- migrations already applied;
- relevant folders and services;
- known limitations.

23.3 3. Phase objective

Use the exact mission from this document.

23.4 4. In scope

List the work packages and mandatory behavior.

23.5 5. Out of scope

List later-phase capabilities so Codex does not improvise them badly or claim them complete.

23.6 6. Non-negotiable architecture

Include:

- shared product rule;
- contracts and versions;

- privacy boundary;
- no game modification;
- immutable published versions;
- mandatory verification gates where relevant;
- feature-flag behavior.

23.7 7. Implementation sequence

Codex should:

1. inspect and report baseline;
2. create or confirm branch;
3. identify impacted contracts and migrations;
4. implement vertical slices;
5. add tests with each slice;
6. run migrations and tests;
7. perform demonstration;
8. update documentation;
9. produce completion report.

23.8 8. Test requirements

Copy the relevant phase test strategy and acceptance checklist.

23.9 9. Required artifacts

Specify schemas, screenshots, reports, sample packages, logs, or recordings.

23.10 10. Prohibited shortcuts

Explicitly prohibit fake persistence, fake progress, one-off hard-coding, silent data retention, and phase-inappropriate automatic behavior.

23.11 11. Completion response

Require:

- summary;
- commits;
- files changed;
- migrations;
- tests;
- demonstration;
- unresolved blockers;
- exit-gate status item by item;
- exact next-phase handoff.

Completion Report Template

Phase:

Implementation date:

Repository:

Branch:

Starting commit:

Ending commit:

Application version:

Build ID:

1. Objective achieved
2. Work packages completed
3. Architecture decisions
4. Database and schema changes
5. API/protocol changes
6. UI and user-flow changes
7. Native/service changes
8. Tests and results
9. Manual demonstration and evidence
10. Performance/security/privacy findings
11. Known limitations
12. Deferred authorized work
13. Rollback instructions
14. Exit-gate checklist
15. Recommendation: PASS / NO-GO

The report must say NO-GO when a mandatory exit item fails. Optimistic punctuation does not alter the gate.

Risk Register

ID	Risk	Probability	Impact	Phase owner	Mitigation
R-01	Shared frontend extraction causes website regression	Medium	High	B-1	Incremental extraction, parity tests, feature flags
R-02	Desktop shell creates security exposure	Medium	High	B-1/B-6	narrow native command allowlist, origin restrictions, audit
R-03	Capture fails in common fullscreen setup	Medium	High	B-2	source-health detection, supported-mode guidance, testing
R-04	Capture consumes too many resources	Medium	High	B-2/B-4	downscale, bounded rate, profiling, backpressure
R-05	Browser pairing can be abused	Low/Medium	Critical	B-2/B-6	loopback, origin approval, session auth, anti-replay
R-06	Creator wizard collects insufficient data	Medium	High	B-3	guided coverage, mandatory negatives, data-health gate

ID	Risk	Probability	Impact	Phase owner	Mitigation
R-07	Natural scenes reconstruct poorly	Medium	High	B-4	better recordings, submaps, planar/context hybrid, retry guidance
R-08	Similar wrong place passes retrieval	Medium	Critical	B-4	hard-negative margin, geometry, pose, temporal veto
R-09	Calibration overfits	Medium	High	B-4	locked tests, separate datasets, reproducibility
R-10	Inference degrades game performance	Medium	High	B-4/B-6	selected frames, precompute, hardware tiers, cancellation
R-11	Duplicate event advances story twice	Low/Medium	Critical	B-5	idempotency and stage-version transaction
R-12	Offline sync corrupts progression	Medium	High	B-5	event log, version conflict handling, tests
R-13	Captain overrides conceal reliability issue	Medium	Medium	B-5	separate classification, required reason, review workflow
R-14	Update changes waypoint behavior	Medium	Critical	B-6	package/model pinning, retest state, rollback
R-15	Runtime package executes unsafe content	Low	Critical	B-6	data-only schema, validation, signing, safe extraction
R-16	External creators cannot complete workflow	Medium	High	B-6	unassisted usability testing and iteration

ID	Risk	Probability	Impact	Phase owner	Mitigation
R-17	Privacy promises are not enforced	Low/Medium	Critical	All	automated retention tests, visible controls, audit
R-18	Game policy changes	Low/Medium	High	B-6/Operations	conservative boundary, current review, kill switch

Phase Metrics

26.1 B-1 metrics

- percentage of Player/Captain/Studio UI shared between web and desktop;
- contract-test coverage;
- lifecycle transition coverage;
- migration success;
- duplicate-event protection.

26.2 B-2 metrics

- capture-session success rate;
- dropped-frame rate;
- scan usable-frame rate;
- capture CPU/GPU usage;
- pairing success and expiration behavior;
- zero unintended Player frame retention.

26.3 B-3 metrics

- creator completion rate;
- average number of unresolved data-health warnings;
- wizard abandonment points;
- proportion of packages schema-valid on first submission;
- unassisted usability completion.

26.4 B-4 metrics

- first-scan positive success;
- success within two guided attempts;
- false accepts in locked negatives;
- pose-localization success;
- ambiguity rate;
- runtime duration;
- package size;
- calibration reproducibility.

26.5 B-5 metrics

- exact-once progression success;
- retry distribution;
- Captain confirmation rate;
- override rate and reasons;
- offline sync success;
- Phase A presentation recovery after interruption.

26.6 B-6 metrics

- installer/update success;
- crash rate;
- hardware support coverage;
- external creator completion;
- support issue categories;
- privacy/security defects;
- package compatibility failures.

Post-Phase-B Roadmap Boundary

The following capabilities may begin only after Phase B is complete or through a separately approved experimental track that cannot destabilize Phase B:

27.1 Phase C candidate: Automatic passive verification

- low-frequency stage-aware observation;
- player consent and visible status;
- no always-on universal recognition;
- automatic trigger only for waypoints validated for passive mode.

27.2 Phase D candidate: View-matched external AR

- select the nearest edited viewpoint;
- transition from current frame into matching scene;
- viewpoint-specific cinematic assets;
- optical-flow-assisted minor motion.

27.3 Phase E candidate: Live surface anchoring

- homography or pose-based message placement;
- continuous rock-carving or symbol alignment;
- external companion display only;
- robust loss/reacquisition behavior.

27.4 Phase F candidate: Held-object transformation

- location verified first;
- held-item detection or segmentation;
- custom Three.js or video artifact;
- external view rather than game modification.

These are not hidden incomplete B-4 tasks. They deliberately build on the verified Phase B foundation.

Appendix A: Canonical Domain Entity Summary

28.1 VisionWaypoint

Represents the stable reusable asset.

Minimum fields:

```
id
owner_id
name
description
waypoint_type
island_or_region
thumbnail_asset_id
share_scope
archived_at
created_at
created_by
updated_at
```

28.2 VisionWaypointVersion

Represents an editable draft or immutable published version.

```
id
waypoint_id
version_number
parent_version_id
state
verification_profile
schema_version
engine_compatibility
model_compatibility
build_input_manifest_id
runtime_package_id
calibration_report_id
published_at
published_by
created_at
```

created_by

28.3 WaypointRecording

id
waypoint_version_id
recording_type
artifact_id
capture_device_id
capture_source_metadata
resolution
aspect_ratio
frame_rate
fov_metadata
environment_tags
quality_summary
creator_label
creator_notes
retention_state
created_at

28.4 WaypointPoseRegion

id
waypoint_version_id
region_type
coordinate_system_id
geometry_payload
is_exclusion
facing_target_id
maximum_facing_angle
minimum_visibility
creator_source
finalized_by_build_id

28.5 WaypointBuildJob

id
waypoint_version_id
state
build_mode
engine_version
model_bundle_version
input_manifest_hash
progress_stage
progress_detail
started_at
completed_at
failure_code

failure_detail

28.6 WaypointVerificationAttempt

id
story_session_id
story_stage_version
waypoint_version_id
operation_mode
companion_device_id
package_hash
engine_version
model_bundle_version
outcome
failure_reasons
quality_summary
evidence_summary
captain_judgment
captain_override_id
started_at
completed_at
sync_state

Appendix B: Canonical Companion Message Catalog

Message type	Direction	Purpose
companion.hello	Companion -> UI	Advertise identity and protocol
companion.capabilities	Companion -> UI	Report capture, hardware, and runtime capabilities
pairing.requested	UI -> Companion	Begin browser pairing
pairing.approved	Companion -> UI	Establish authenticated session
pairing.revoked	Either	End trust relationship
capture.targets_requested	UI -> Companion	List/selectable sources through approved native flow
capture.target_selected	Companion -> UI	Confirm source and metadata
creator_recording.started	UI -> Companion	Begin retained creator recording
creator_recording.progress	Companion -> UI	Real capture and quality status
creator_recording.completed	Companion -> UI	Return artifact and metadata
player_scan.started	UI -> Companion	Begin transient scan
player_scan.progress	Companion -> UI	Return real scan progress
player_scan.completed	Companion -> UI	Return evidence-session reference
verification.requested	UI/Coordinator -> Vision worker	Evaluate evidence against package
verification.progress	Vision worker -> UI/Coordinator	Optional truthful processing state
verification.completed	Vision worker -> UI/Coordinator	Return final outcome and evidence summary
vision.paused	Either	Pause capture/processing
health.status	Companion -> UI	Heartbeat and current state
diagnostic.bundle_requested	UI -> Companion	Create privacy-safe diagnostic package
diagnostic.bundle_available	Companion -> UI	Return metadata/path/reference

Appendix C: Verification Result Example

```
{
  "schemaVersion": "1.0",
  "attemptId": "attempt_01K...",
  "waypointVersionId": "vpv_01K...",
  "storySessionId": "story_session_01K...",
  "storyStageVersion": 12,
  "outcome": "VERIFIED",
  "operationMode": "SHADOW",
  "automaticProgressionAllowed": false,
  "captureQuality": {
    "capturedFrames": 55,
    "usableFrames": 13,
    "selectedFrames": 9,
    "motionStatus": "GOOD_SLOW_SCAN"
  },
  "retrieval": {
    "bestTargetReference": "ref_014",
    "bestTargetScore": 0.91,
    "bestNegativeReference": "neg_006",
    "bestNegativeScore": 0.74,
    "separationMargin": 0.17,
    "passed": true
  },
  "geometry": {
    "localizedFrames": 7,
    "medianInliers": 61,
    "medianReprojectionErrorPx": 2.1,
    "spatialCoverage": 0.63,
    "passed": true
  },
  "pose": {
    "acceptedRegion": true,
    "facingTarget": true,
    "targetVisibility": 0.58,
    "passed": true
  },
  "temporal": {
```

```
    "agreement": 0.92,  
    "trajectoryPlausible": true,  
    "passed": true  
  },  
  "checkpointRule": {  
    "type": "EXACT_LANDMARK",  
    "passed": true  
  },  
  "ambiguityVeto": false,  
  "failedGates": [],  
  "engine": {  
    "engineVersion": "0.4.0",  
    "modelBundle": "vision-models-2026.07",  
    "executionProvider": "CUDA"  
  }  
}
```

The numbers shown are illustrative and not default thresholds.

Appendix D: Runtime Package Manifest Example

```
{
  "schemaVersion": "1.0",
  "waypointId": "vp_01KSBMR7",
  "waypointVersionId": "vpv_01KSBMR7V3",
  "versionNumber": 3,
  "name": "Sailor's Bounty Message Rock",
  "type": "EXACT_LANDMARK",
  "verificationProfile": "STORY_CRITICAL",
  "engineCompatibility": {
    "minimum": "0.4.0",
    "maximumExclusive": "1.0.0"
  },
  "modelCompatibility": {
    "family": "vision-models-2026",
    "minimum": "2026.07"
  },
  "interaction": {
    "mode": "HOLD_TO_SCAN",
    "targetDurationMs": 5000,
    "minimumDurationMs": 3000,
    "maximumDurationMs": 7000
  },
  "artifacts": [
    {"path": "reference-global-index.bin", "sha256": "...", "size": 12345},
    {"path": "reference-local-features.bin", "sha256": "...", "size": 45678},
    {"path": "sparse-map.bin", "sha256": "...", "size": 98765},
    {"path": "hard-negative-index.bin", "sha256": "...", "size": 23456},
    {"path": "verification-rules.json", "sha256": "...", "size": 3456}
  ],
  "calibrationReportHash": "...",
  "publishedAt": "2026-07-17T23:00:00-04:00",
  "publishedBy": "creator_..."
}
```

Appendix E: Phase Readiness Checklist

32.1 Before beginning B-1

- ☐ canonical repository confirmed;
- ☐ current working website backed up and testable;
- ☐ governing documents committed;
- ☐ current story and Studio architecture inventoried;
- ☐ database migration process understood;
- ☐ branch strategy agreed.

32.2 Before beginning B-2

- ☐ B-1 gate passed;
- ☐ native framework ADR approved;
- ☐ Companion protocol schema stable enough for implementation;
- ☐ privacy data-flow diagram reviewed;
- ☐ target Windows versions identified;
- ☐ capture technical spike completed.

32.3 Before beginning B-3

- ☐ B-2 capture sessions stable;
- ☐ recording artifacts and metadata available through contracts;
- ☐ draft persistence and versioning confirmed;
- ☐ design system supports wizard and library patterns;
- ☐ creator workflow wireframes reviewed against governing specification.

32.4 Before beginning B-4

- ☐ B-3 build-input package validates;
- ☐ pilot locations selected;
- ☐ positive and negative recordings collected through Studio;
- ☐ model/library ADRs approved;
- ☐ hardware profiling environment ready;
- ☐ locked-test policy established.

32.5 Before beginning B-5

- ☐ B-4 three-pilot gate passed;
- ☐ shadow result contract stable;
- ☐ Phase A presentation events documented;
- ☐ story idempotency design reviewed;
- ☐ first production candidate selected;
- ☐ Captain UX reviewed.

32.6 Before beginning B-6

- ☐ B-5 end-to-end story demonstration passed;
- ☐ no critical known false accept remains;
- ☐ installer/update architecture selected;
- ☐ external usability cohort recruited;
- ☐ security/privacy review plan approved;
- ☐ release-channel and rollback plan prepared.

Appendix F: Master Definition of Done

Phase B is not complete unless every applicable item below is satisfied.

33.1 Product

- ☐ Creators can author Vision Waypoints without Codex.
- ☐ Waypoints are reusable and versioned.
- ☐ Players can perform hold-to-scan verification.
- ☐ Captain can monitor and recover.
- ☐ Correct verification triggers Phase A presentation.
- ☐ Incorrect and ambiguous scans do not reveal story content.
- ☐ Website, PWA, and desktop are one shared product.

33.2 Architecture

- ☐ Platform adapters isolate native differences.
- ☐ Protocols and package schemas are versioned.
- ☐ Published versions are immutable.
- ☐ Models are replaceable behind adapters.
- ☐ Runtime packages are data-only and safe.
- ☐ Offline state uses idempotent events.

33.3 Reliability

- ☐ Mandatory gates are implemented.
- ☐ Hard-negative veto is implemented.
- ☐ 3D or governed geometric localization is used where required.
- ☐ Several frames must agree.
- ☐ Calibration is per waypoint.
- ☐ Locked positive and negative tests exist.
- ☐ Automatic mode promotion is evidence-based and revocable.

33.4 Privacy and safety

- ☐ Player frames are transient by default.
- ☐ Capture source and status are visible.
- ☐ Pairing is secure and revocable.

- ☐ No injection, memory reading, game modification, or input automation exists.
- ☐ Diagnostic retention is opt-in.
- ☐ Creator recordings can be deleted.
- ☐ Package and update integrity are checked.

33.5 Delivery

- ☐ Installer and updater work.
- ☐ Rollback works.
- ☐ Hardware support is documented.
- ☐ Documentation exists for all roles.
- ☐ External creators complete the flow.
- ☐ Support diagnostics are privacy-safe.
- ☐ Governing and phase completion reports are current.

Appendix G: Glossary

Accepted pose region - The set of camera positions and orientations from which a waypoint may pass.

Ambiguity veto - A mandatory rejection when target and negative evidence are too close or subsystems disagree materially.

Build-input package - The Studio-generated, schema-valid description of recordings, regions, masks, and settings submitted to the build engine.

Captain Confirmed mode - The engine recommends an outcome, but Captain approval is required before story progression.

Capture Companion - The local native service that captures the selected game window, manages recordings and scans, and runs or connects to local vision inference.

Hard negative - A wrong location, object, pose, or view that resembles the target and is explicitly used to prevent false acceptance.

HLoc-inspired - A hierarchical localization approach using broad retrieval followed by local matching and pose localization. It describes architecture, not a permanent dependency.

Immutable published version - A waypoint release whose package, calibration, and rules cannot be silently edited after publication.

Local feature - A stable point or region descriptor used to match physical scene detail.

Operation mode - Development, Shadow, Captain Confirmed, or Automatic behavior governing how verification affects the story.

Phase A - The existing passive presentation capability that displays prepared images, video, audio, Three.js scenes, and transitions.

Phase B - The six-phase program defined in this roadmap that adds reusable guided active visual verification and desktop delivery.

PWA - Installable Progressive Web App form of the website.

Runtime package - The compact, versioned, checksummed waypoint data used by the local verifier; it excludes raw creator recordings.

Shadow mode - The engine evaluates scans and records recommendations without advancing the story automatically.

Spatial coverage - Measurement that valid evidence is distributed across enough of the target or scene rather than clustered on one repeated texture.

Temporal consistency - Agreement of several frames and a physically plausible camera trajectory during one scan.

Vision Waypoint - A reusable Studio asset that defines a visually verifiable location, landmark, viewpoint, object, item, or sequence.

Final Governing Statement

Implementation Mandate

Phase B SHALL be delivered as six substantial, stable, demonstrable phases. Codex is responsible for implementing the platform and tools, not for becoming a permanent manual waypoint-construction service. The website, PWA, and Windows desktop application SHALL remain one shared product. Verification SHALL remain multi-frame, multi-gate, hard-negative-aware, geometry- and pose-conscious, ambiguity-rejecting, locally private by default, and recoverable through Captain oversight. No phase may be declared complete while its mandatory exit evidence is missing.

That structure is intentionally strict because a story system that reacts to the wrong rock is not charmingly imperfect. It is simply a very expensive rock enthusiast.